

Fondamenti Logici dell'Informatica (Modulo 1)

Formalismo di calcolo

Un **formalismo di calcolo** è un formalismo che risponde alla domanda “cosa vuol dire calcolare”

Un formalismo è una descrizione matematicamente rigorosa di un fenomeno, in genere ottenuta tramite manipolazione di espressioni simboliche.

Esistono numerosi formalismi di calcolo:

- MdT
- Sistemi di Post
- funzioni primitive ricorsive con operatore di minimizzazione
- Random Access Machines
- Linguaggi di Programmazione rigorosamente specificati

Tesi di Church-Turing

Ogni funzione calcolabile da un formalismo di calcolo sufficientemente espressivo è calcolabile da una macchina di Turing e viceversa.

Formalismi equivalenti in **cosa calcolano**, la differenza risiede nei punti di forza o debolezza di questi formalismo (esempio con linguaggi di programmazione diversi).

Macchine di Turing

- calcolo $\rightarrow$ modifica di un nastro con celle discrete contenenti una quantità **finita** di informazioni
- operazioni **locali** $\rightarrow$ spostamento testina a dx o sx di una sola posizione a ogni passo

λ -calcolo (Church)

- calcolo $\rightarrow$ semplificazione di espressioni
- espressioni $\rightarrow$ tutto è **funzione** $\rightarrow$ IN = funzioni, OUT = funzioni

Confronto tra MdT e λ -calcolo

Confronto computazionale

MdT	λ -calcolo
passo = $O(1)$ (tempo)	passo con implementazione naif: $O(n^2)$
passo = $O(1)$ (spazio)	passo con implementazione efficiente: $O(?)$ $\rightarrow$ complesso da studiare

MdT	λ -calcolo
Ottimo per studio complessità	Pessimo per studio complessità

Confronto caratteristiche

MdT	λ -calcolo
Imperativo (diverso da linguaggi imperativi)	Fulcro di tutti i linguaggi funzionali
Non composizionale $\rightarrow$ le MDT consumano l'intero input per dare una risposta e necessitano spesso di simboli aggiuntivi	Composizionale
Basso livello	Alto livello
Implementazione Difficile	Implementazione Facile
Pessimo per studio di linguaggi di programmazione	Ottimo per studio di linguaggi di programmazione $\rightarrow$ consente di scegliere scientificamente quali costrutti implementare per ottenere un buon linguaggio di programmazione

Costo di un passo di β -riduzione Si consideri un generico passo di β -riduzione:

$$(\lambda x.M)N \rightarrow_{\beta} M\{N/x\}$$

Per effettuare questa operazione il costo computazionale sarà: $O(|M| + |M|_x \cdot |N|)$, dove $|M|_x$ è il numero di occorrenze di x in M . Nel caso pessimo si potrebbe avere:

- N di lunghezza pari o simile a M
- M costituito esclusivamente da applicazioni di x

$$O(|M| + |M|_x \cdot |N|) = O(|M| + \frac{|M|}{2} \cdot |M|) = O(|M|^2) = O(t^2)$$

dove t è la lunghezza del λ -termine iniziale.

MdT

Definizione: MdT

Una MdT è definita da una tupla (A, Q, q_0, q_f, δ) dove:

- A è un **alfabeto**, insieme non vuoto e finito di simboli
- Q è un insieme finito e non vuoto di **stati**
- q_0 è lo stato **iniziale**
- q_f è lo stato **finale**
- δ è la **funzione di transizione**: $\delta : Q \times A \rightarrow Q \times A \times \{L, R\}$

Definizione: Stato di una MdT Lo stato di una MdT è una tripla (α, i, q) tale che:

- il **nastro infinito** α è una funzione da $\mathbb{Z}$ ad A . $\alpha(k) = a \iff$ k-esima cella del nastro contiene il simbolo a
- $i \in \mathbb{Z}$ è la **posizione della testina** sul nastro.
- $q \in Q$ è lo **stato corrente**.

Esecuzione di una MdT

Una MdT (A, Q, q_0, q_f, δ) in uno stato (α, i, q) non finale transisce in un nuovo stato (α', i', q') se:

- $\delta(\alpha(i), q) = a, q', x \rightarrow$ la testina legge il contenuto di $\alpha(i)$ della cella corrente i , lo stato corrente si aggiorna da q a q'
- $\alpha'(i) = a$ e $\alpha'(n) = \alpha(n)$ per $n \neq i$
- $i' = i + 1$ se $x = R$, altrimenti $i' = i - 1$ se $x = L$

λ -calcolo

Sintassi

Tutto è una funzione unaria anonima:

$$t ::= x \mid tt \mid \lambda x.t$$

dove:

- t è detto **termine**
- si utilizzeranno $t, s, u, M, N, \dots$ per indicare un termine
- $x, y, z, w, \dots$ per indicare l'**occorrenza di una variabile**
- $t_1 t_2$ è la **chiamata di funzione** $\rightarrow t_1$ riceve in input la funzione t_2 . Per disambiguare si fa uso di parentesi tonde (notazione matematica classica). Esempio: $t_1(t_2)$.
- $\lambda x.t$ viene detta **astrazione** e consiste in una **funzione anonima** il cui **parametro formale** è x e il cui **corpo** è t . Notazione: $x \rightarrow t$ oppure $f(x) = t$ se la funzione ha nome f . Con un'astrazione il corpo t diventa lo scope della variabile legata x .

Regole di precedenza e associatività

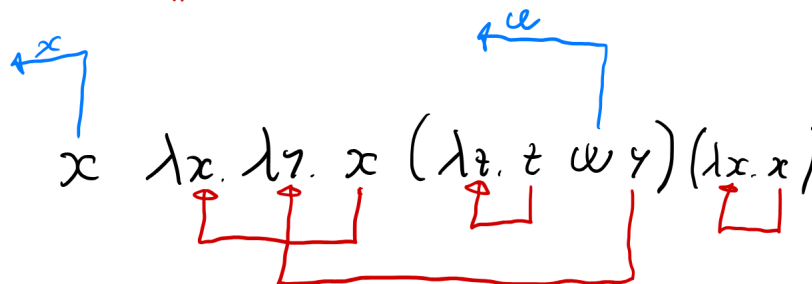
- **applicazione** ha precedenza su **astrazione**, segue che $\lambda x.xx$ si legge $\lambda x.(xx)$

- **applicazione** è associativa a sinistra: xyz si legge come $(xy)z$.
L'associatività a sinistra permette di "simulare" meglio una funzione n -aria.

Diagramma di legame

DIAGRAMMA DI LEGAME:

— OGNI VARIABILE LEGATA È COLLEGATA AL SUO BINDO, IL PIÙ INTERNO



— OGNI VARIABILE LIBERA PUNTA "FUORI" CON IL SUO NOME

Figure 1: Diagramma di legame

Funzioni n -arie e Applicazione parziale

Una funzione binaria $f(x, y) = g(x, y)$ può essere vista come una funzione unaria che restituisce una funzione unaria: $\lambda x. \lambda y. gxy$

Si osserva che $g(x, y)$ viene codificato con il passaggio sequenziale di due input (funzioni unarie) $\rightarrow gxy$ che equivale a $(gx)y$. Significa che è possibile applicare le funzioni parzialmente. Ad esempio $(\lambda x. \lambda y. x + y)2$ riduce alla funzione $\lambda y. 2 + y$.

Intuizione: Riduzione

Un λ -termine può ridurre a un altro λ -termine t' rimpiazzando una chiamata di funzione $(\lambda x. M)N$ con il corpo M dove sostituisco x con N .

Esempio: $(\lambda x. yx)(zz)$ riduce a $y(zz)$.

Una definizione rigorosa della nozione di sostituzione è fondamentale.

Variabili libere e legate

$$t ::= x \mid tt \mid \lambda x.t$$

I nomi dati ai parametri formali non sono rilevanti nella definizione di una funzione: $\lambda x.x$ è la stessa funzione di $\lambda y.y$

Ciò cambia con i nomi delle variabili globali: $\lambda x.y$ e $\lambda x.z$ sono due programmi diversi. Intuitivamente è impossibile rimpiazzare l'uno con l'altro in un contesto dove $y = 0$ e $z = 1$ senza ottenere risultati diversi.

Nell'astrazione $\lambda x.t$, λ viene detto **binder** poiché lega la variabile x al corpo t . Una variabile non legata viene detta **libera**. Le variabili legate sono parametri formali, quelle libere sono tutte variabili globali.

Definizione: insieme delle variabili libere L'insieme delle variabili libere $FV(t)$ di t si calcola come segue:

- $FV(x) = \{x\}$
- $FV(MN) = FV(M) \cup FV(N)$
- $FV(\lambda x.M) = FV(M) \setminus \{x\}$

Esempio: $FV(\lambda x.xy(\lambda y.yz)) = \{y, z\}$

Definizione: α -conversione $\equiv_\alpha$

Due λ -termini si dicono α -convertibili ($t_1 \equiv_\alpha t_2$) se è possibile ottenere l'uno dall'altro, effettuando una ridenominazione delle sole variabili legate tale che le occorrenze legate di una variabile in posizione corrispondente nei due termini siano legate dai binder in posizione corrispondente e che le occorrenze di variabili libere abbiano in posizione corrispondente una variabile libera con lo stesso nome.

Più semplicemente sono due λ -termini che hanno il medesimo diagramma di legame.

Alcuni esempi:

- $\lambda x.\lambda y.xyz \equiv_\alpha \lambda y.\lambda w.ywz$
- $\lambda x.\lambda y.xyz \not\equiv_\alpha \lambda x.\lambda z.xzz$
- $\lambda x.\lambda y.xyz \not\equiv_\alpha \lambda x.\lambda y.yxz$
- $\lambda x.\lambda y.xyz \not\equiv_\alpha \lambda y.\lambda w.xyw$

Attenzione!! : termini α -equivalenti saranno considerati uguali. L' α -equivalenza è una **classe di equivalenza**

Questa operazione consente di mantenere un diagramma di legame, evitando catture.

Sostituzione

Quando si sostituisce in M un termine N al posto di una variabile x ($M\{N/x\}$) occorre prestare attenzione a non effettuare catture accidentali delle variabili libere.

Esempio: $(\lambda x.xy)\{zz/y\} = \lambda x.x(zz)$ ma $(\lambda x.xy)\{xx/y\} \neq \lambda x.x(zz)$, questo perché le x a sinistra in xx sono globali, ma a destra sono parametri formali (dx e sx di $\neq$)

La semantica è quindi errata. Per evitare questa problematica si usa α -conversione $\rightarrow$ sempre possibile grazie all'esistenza di infinite variabili.

Soluzione tramite α -conversione: $(\lambda x.xy)\{xx/y\} \equiv_\alpha (\lambda z.zy)\{xx/y\} = \lambda z.z(xx)$

Questo indica che spesso per poter sostituire, devo prima α -convertire.

Definizione $M\{N/x\}$ è definito come:

- $x\{N/x\} = N$
- $y\{N/x\} = y$
- $(t_1 t_2)\{N/x\} = t_1\{N/x\} t_2\{N/x\}$
- $(\lambda x.M)\{N/x\} = \lambda x.M \rightarrow$ tutte le x in M sono legate, quindi non libere e non sostituibili.
- $(\lambda y.M)\{N/x\} = \lambda z.M\{z/y\}\{N/x\}$ per $z \notin \text{FV}(M) \cup \text{FV}(N)$

Terminologia: z è **sufficientemente fresca** se $z \notin \text{FV}(M) \cup \text{FV}(N)$ e **fresca** se non è mai stata utilizzata prima. Ogni variabile fresca è anche sufficientemente fresca.

Definizione: β -riduzione

$t_1 \beta$ -riduce a t_2 **in un passo** (Notazione: $t_1 \rightarrow_\beta t_2$) $\iff$ ottengo t_2 da t_1 rimpiazzando da qualche parte in t_1 il **redex** $(\lambda x.M)N$ con il suo **ridotto** $M\{N/x\}$.

Esempio: $\lambda x(\lambda y.xy)x \rightarrow_\beta \lambda x.zx$ dove è stato ridotto il *redex* $(\lambda y.xy)x$

La definizione formale avviene tramite un sistema di inferenza, composto dalle seguenti regole:

$$\overline{(\lambda x.M)N \rightarrow_\beta M\{N/x\}}$$

$$\frac{M \rightarrow_\beta M'}{MN \rightarrow_\beta M'N}$$

$$\frac{M \rightarrow_\beta M'}{NM \rightarrow_\beta NM'}$$

$$\frac{M \rightarrow_{\beta} M'}{\lambda x.M \rightarrow_{\beta} \lambda x.M'}$$

L'ultima regola modifica il corpo a runtime $\rightarrow$ grossa differenza con linguaggi compilati $\rightarrow$ l'ottimizzazione del codice avviene durante la compilazione. Il corpo di una funzione può essere ridotto prima dell'invocazione.

Definizione: β -riduzione in n passi

- $t \rightarrow_{\beta}^0 t$
- $t \rightarrow_{\beta}^{n+1} t''$ sse $t \rightarrow_{\beta} t'$ e $t' \rightarrow_{\beta}^n t''$

Definizione: chiusura riflessiva e transitiva della β -riduzione

- $t \rightarrow_{\beta}^* t' \iff \exists n. t \rightarrow_{\beta}^n t'$

Forme normali

- t è una **forma normale** ($t \nrightarrow_{\beta}$) sse $\nexists t'. t \rightarrow_{\beta} t'$
- t ha una **forma normale** t' sse $t \rightarrow_{\beta}^* t' \wedge t' \nrightarrow_{\beta}$
- t ha una **forma normale o può convergere** sse esiste un t' tc t ha forma normale t' .

Non determinismo

La relazione $\rightarrow_{\beta}$ è **non deterministica**, ovvero ci sono dei t tc esistono t_1, t_2 distinti tc $t_1 \leftarrow_{\beta} t \rightarrow_{\beta} t_2$

Strade diverse hanno lunghezze diverse, ma comunque **convergenti** $\rightarrow$ è una proprietà del λ -calcolo.

Questa proprietà è nota anche come **unicità della forma normale** e permette di ignorare l'ordine di esecuzione delle due "strade". Questo introduce un vantaggio notevole, infatti i linguaggi che non godono di questa proprietà e consentono di introdurre *side effects* durante la valutazione di un'espressione, possono dare origine a comportamenti inattesi.

Espressività del λ -calcolo

Turing completezza $\iff$ **tipi di dato, scelta, ripetizione.**

Ricorsione nel λ -calcolo

Qui si ricorre alla logica per andare ad esplorare eventuali meccanismi che consentono di ottenere ragionamenti circolari (cicli).

Paradosso di Russell Il λ -calcolo consente di ottenere il paradosso di Russell:

- predicato binario $\rightarrow$ applicazione
- è possibile fare auto-applicazione
- negazione tramite un λ -termine qualsiasi
- trasformazione di una proprietà (predicato) in un insieme auto-applicabile (assioma inconsistente di comprensione). $\rightarrow$ tramite binding della variable ($Y|Y \dots$ è un binding a tutti gli effetti).

$$X = \{Y | (Y \notin Y)\} \rightarrow \lambda Y. g(YY)$$

La duplicazione YY consente quindi di avere un programma che duplica il suo codice. Si ha quindi con g funzione identità il più piccolo λ -termine divergente. Questo lascia aperta la possibilità che il λ -calcolo sia Turing completo (**TH: linguaggio Turing completo è divergente**):

$$(\lambda x. xx)(\lambda x. xx) \rightarrow_{\beta} (\lambda x. xx)(\lambda x. xx) \rightarrow_{\beta} \dots$$

Divergenze Un termine t_0 può divergere sse $\forall i. \exists t_{i+1}. t_i \rightarrow_{\beta} t_{i+1}$ Un termine può divergere e convergere allo stesso tempo (per via del non determinismo).

Punto fisso

Dato una funzione $f : A \rightarrow A$, $x \in A$ è un punto fisso di f se $f(x) = x$. Esistono funzioni che non ammettono punti fissi, tuttavia non è così nel λ -calcolo.

Punto fisso in λ -calcolo

Un λ -termine t è un punto fisso per f se $ft =_{\beta} t$ dove $=_{\beta}$ è la chiusura simmetrica, riflessiva e transitiva di $\rightarrow_{\beta}$.

I punti fissi esistono sempre: $(\lambda x. f(xx))(\lambda x. f(xx))$ è un punto fisso di f .

Questo risultato è dato dalla divergenza che in matematica non esiste, in quanto non si ha il concetto di calcolo. Intuitivamente si ha perché un “termine divergente + 1” rimane un termine divergente.

Operatore di punto fisso

Un λ -termine Y è un **operatore di punto fisso** sse per ogni λ -termine f , Yf è un punto fisso di f . Si ha quindi un programma universale che dato un termine, ne calcola il punto fisso.

Esempio: $\lambda f. ((\lambda x. f(xx))(\lambda x. f(xx)))$. La presenza di questo operatore è ciò che consente di codificare la ricorsione.

Codifica delle funzioni ricorsive

Le funzioni ricorsive esplicite non esistono in λ -calcolo, perciò si trasforma una funzione ricorsiva in un **funtore non ricorsivo**, utilizzando il punto fisso di questo funtore.

Metodo per realizzare ricorsione in λ -calcolo: prendo la definizione ricorsiva, aggiungo λf davanti (nome funzione) e passo tutto in input a un operatore di punto fisso.

Esempio fattoriale:

```
let fact =  
   $\lambda n.$   
    match  $n$  with  
    | 0  $\rightarrow$  (S0)  
    | (Sm)  $\rightarrow$  (Sm)  $\cdot$  fact( $m$ )
```

diventa:

```
let fact =  
  Y  
  ( $\lambda$ fact.  
     $\lambda n.$   
      match  $n$  with  
      | 0  $\rightarrow$  (S0)  
      | (Sm)  $\rightarrow$  (Sm)  $\cdot$  fact( $m$ ))
```

Si ottiene la ricorsione tramite duplicazione del codice. La codifica semplice della circolarità tramite ricorsione, mostra ulteriormente la natura funzionale del λ -calcolo.

Esempio di invocazione Considero

$$F = (\lambda \text{fact}.\lambda n.\text{match } n \text{ with } | 0 \rightarrow (S0) | (Sm) \rightarrow (Sm) \cdot \text{fact}(m))$$

Applico l'operatore di **punto fisso**:

$$\begin{aligned} \text{fact } 4 &\equiv (YF)4 \equiv (\lambda f.((\lambda x.f(xx))(\lambda x.f(xx)))F)4 \\ &\rightarrow_{\beta} ((\lambda x.F(xx))(\lambda x.F(xx)))4 \\ &\rightarrow_{\beta} (F((\lambda x.F(xx))(\lambda x.F(xx))))4 \equiv (F(YF))4 \end{aligned}$$

Proseguo con le β -riduzioni.

$$\begin{aligned}
(F(YF))4 &\equiv ((\lambda \text{fact}.\lambda n.\text{match } n \text{ with } \dots)(YF))4 \\
&\rightarrow_{\beta} (\lambda n.\text{match } n \text{ with} \\
&\quad | 0 \rightarrow (S0) \\
&\quad | (Sm) \rightarrow (Sm) \cdot YF(m))4 \\
&\rightarrow_{\beta} \text{match } 4 \text{ with} \\
&\quad | 0 \rightarrow (S0) \\
&\quad | (Sm) \rightarrow (Sm) \cdot YF(m) \\
&= 4 \cdot (YF)(3) \\
&= 4 \cdot 3 \cdot (YF)(2) \\
&= 4 \cdot 3 \cdot 2 \cdot (YF)(1) \\
&= 4 \cdot 3 \cdot 2 \cdot 1 \\
&= 24
\end{aligned}$$

Tipi di dato algebrici

Alcuni: esempi:

$$\text{type B} = \text{true: B} \mid \text{false: B}$$

$$\text{type N} = 0: \text{N} \mid \text{S: N} \rightarrow \text{N}$$

S è la funzione “successore” che prende in input un numero n e restituisce il successore di n .

$$\text{type List T} = []: \text{List T} \mid (::) : \text{T} \rightarrow \text{List T} \rightarrow \text{List T}$$

$((::)$ è la testa della lista)

$$\text{type Seed} = \text{Hearts: Seed} \mid \text{Clubs: Seed} \mid \text{Flowers: Seed} \mid \text{Pikes: Seed}$$

Gli elementi di un tipo algebrico sono tutti distinti.

$$\text{type Option_N} = \text{None: Option_N} \mid \text{Some: N} \rightarrow \text{Option_N}$$

Tipo di dato algebrico che codifica un albero, in cui il tipo delle foglie differisce da quello dei nodi non foglia.

$$\text{type Tree K V} = \text{Node: K} \rightarrow \text{Tree K V} \rightarrow \text{Tree K V} \rightarrow \text{Tree K V} \mid \text{Leaf: V} \rightarrow \text{Tree K V}$$

Costrutto di Pattern Matching

Esempio: funzione dato un albero di tipo *Tree*, vuole sommare tutti i numeri presenti nell'albero.

```
let rec sum: Tree Z Z → Z =  
  λt.  
    match t with  
    | Leaf n → n  
    | Node (k, t1, t2) → k + sum t1 + sum t2
```

In questo caso “match” è l'operatore di pattern matching, serve per rilevare se l'istanza corrente dell'albero è nodo o foglia. Il pattern matching è possibile con i tipi di dato algebrici in quanto, ogni elemento del tipo di dato ha una forma **distinta**. Ogni ramo dell'operatore “match” è un **pattern**.

Si vuole andare a codificare i tipi algebrici nel λ -calcolo, per poi andare ad implementare un operatore di pattern matching che consenta di usare questi tipi. Infine serve dimostrare e verificare la correttezza del tutto.

λ -calcolo: implementazione dei tipi algebrici tramite funzioni

Tipo di dato astratto

Si tratta di un tipo, per cui mi aspetto di avere il tipo stesso e delle precise operazioni. Esempio con stack:

```
type S  
  Empty: () → S  
  Push: S → K → S  
  Pop: S → K × S
```

In questa definizione, ho solo la firma dei metodi associati al tipo, ma non ho alcuna implementazione che conferisca semantica. Prima dell'implementazione occorre dare dei vincoli per un corretto funzionamento delle funzioni, ciò avviene tramite equazioni. L'idea di funzione sembra alla base del tipo di dato astratto, quindi si può provare ad utilizzare le funzioni stesse per dare anche l'implementazione.

1. Codifica di ogni AlgDT con un ADT

Esempio: “not” su booleani

() è il tipo unit

ADT B

true: () $\rightarrow$ B

false: () $\rightarrow$ B

match: $\forall T. B \rightarrow T \rightarrow T \rightarrow T$ dove T è tipo polimorfo

let not =

$\lambda b.$

match b with

| True $\rightarrow$ False

| False $\rightarrow$ True

Equazioni di correttezza su ADT B:

$$\text{match } (\text{true}()) \text{ t f} \rightarrow_{\beta}^* \text{tmatch } (\text{false}()) \text{ t f} \rightarrow_{\beta}^* \text{f}$$

Codice “not” in λ -calcolo:

$$\text{not} = \lambda b. \text{ match b (false()) (true())}$$

Invocazione:

not true

Esempio: AlgDT Tree K V

ADT Tree K V

leaf: $V \rightarrow \text{Tree K V}$

node: $K \rightarrow \text{Tree K V} \rightarrow \text{Tree K V} \rightarrow \text{Tree K V}$

match: $\forall T. \text{Tree K V} \rightarrow$

$(K \rightarrow \text{Tree K V} \rightarrow \text{Tree K V} \rightarrow T) \rightarrow$ (pattern matching nodo)

$(V \rightarrow T) \rightarrow$ (pattern matching foglia)

T (tipo di ritorno)

Equazioni di correttezza su ADT Tree K V:

$$\text{match}_{\text{Tree}}(\text{Node K T}_1 \text{ T}_2)(\lambda k. \lambda t_1. \lambda t_2. n)(\lambda v. l) \rightarrow_{\beta}^* (\lambda k. \lambda t_1. \lambda t_2. \lambda n) K \text{ T}_1 \text{ T}_2$$

L'equazione soprastante è un caso specifico della generalizzazione seguente:

$$\text{match}_{\text{Tree}}(\text{Node } K \ T_1 \ T_2) \ n \ l \rightarrow_{\beta}^* n \ K \ T_1 \ T_2$$

$$\text{match}_{\text{Tree}}(\text{Leaf } V) \ n \ l \rightarrow_{\beta}^* l \ V$$

Codice “sum” in λ -calcolo (con zucchero sintattico):

$$\text{let rec sum} = \lambda t. \text{match}_{\text{Tree}} \ t \ (\lambda k. \lambda t_1. \lambda t_2. k + \text{sum } t_1 + \text{sum } t_2) \ (\lambda v. v)$$

Invocazione:

$$\text{sum Node } K \ T_1 \ T_2$$

2. Implementazione dell'ADT

La funzione “match” viene implementata tramite la funzione identità $(\lambda x.x)$.

Esempio con booleani:

$()$ è il tipo unit

ADT B

true: $() \rightarrow B$

false: $() \rightarrow B$

match: $\forall T. B \rightarrow T \rightarrow T \rightarrow T$ dove T è tipo polimorfo

$$\forall T. B \rightarrow T \rightarrow T \rightarrow T \equiv B \rightarrow (\forall T. T \rightarrow T \rightarrow T)$$

Dal momento che voglio implementare pattern matching tramite identità, il tipo dei booleani sarà definito come segue:

$$B = \forall T. T \rightarrow T \rightarrow T$$

I valori True e False, avranno le seguenti forme:

$$\text{true} : \forall T. T \rightarrow T \rightarrow T$$

$$\text{false} : \forall T. T \rightarrow T \rightarrow T$$

Dal momento che l'operatore di pattern matching sarà implementato con l'identità, ci si aspetta il seguente comportamento:

$$\begin{aligned}\text{match}_B \text{ true } t \text{ f} &\rightarrow_\beta \text{ true } t \text{ f} \rightarrow_\beta^* t \\ \text{match}_B \text{ false } t \text{ f} &\rightarrow_\beta \text{ false } t \text{ f} \rightarrow_\beta^* f\end{aligned}$$

Costruzione dei valori True e False con sole funzioni in λ -calcolo:

$$\begin{aligned}\text{true} &= \lambda t. \lambda f. t \\ \text{false} &= \lambda t. \lambda f. f\end{aligned}$$

Andando a β -ridurre, si nota come con le implementazioni fornite per i booleani e il pattern matching come identità, si ottenga il comportamento corretto da parte dell'operatore di pattern matching:

$$\text{match}_B \text{ true } t \text{ f} = (\lambda x. x)(\lambda t. \lambda f. t) t f \rightarrow_\beta (\lambda t. \lambda f. t) t f \rightarrow_\beta (\lambda f. t) f \rightarrow_\beta t$$

$$\text{match}_B \text{ false } t \text{ f} = (\lambda x. x)(\lambda t. \lambda f. f) t f \rightarrow_\beta (\lambda t. \lambda f. f) t f \rightarrow_\beta (\lambda f. f) f \rightarrow_\beta f$$

True e False non sono più dati passivi, ma sono l'implementazione del costrutto if-then-else. Il dato diventa la risposta all'osservazione del dato stesso $\rightarrow$ non è il pattern matching che effettua la decisione, è il tipo.

Implementazione per il tipo Tree

La definizione seguente del tipo Tree è data dal fatto che anche per questo tipo di dato implementiamo il pattern matching come funzione identità.

$$\text{Tree } K \text{ V} = \forall T. (K \rightarrow \text{Tree } K \text{ V} \rightarrow \text{Tree } K \text{ V} \rightarrow T) \rightarrow (V \rightarrow T) \rightarrow T$$

Un albero è qualcosa che posso interrogare per dirgli “esegui questa cosa se sei un nodo o esegui l'altra se sei una foglia”.

I valori avranno quindi la seguente forma:

$$\text{Node} = \lambda k. \lambda t_1. \lambda t_2. \lambda n. \lambda l. n k t_1 t_2$$

I primi tre elementi sono l'input al costruttore di un nodo. Andiamo a β -ridurre per osservare come l'implementazione del pattern matching tramite funzione identità consente di ottenere il comportamento atteso.

$$\begin{aligned}
& \text{match}_{\text{Tree}}(\text{node } K \ T_1 \ T_2) \ N \ L \rightarrow_{\beta} \\
& (\lambda x.x)((\lambda k.\lambda t_1.\lambda t_2.\lambda n.\lambda l.nkt_1t_2) \ K \ T_1 \ T_2) \ N \ L \rightarrow_{\beta} \\
& ((\lambda k.\lambda t_1.\lambda t_2.\lambda n.\lambda l.nkt_1t_2) \ K \ T_1 \ T_2) \ N \ L \rightarrow_{\beta} \\
& (\lambda n.\lambda l.nK \ T_1 \ T_2) \ N \ L \rightarrow_{\beta} \\
& N \ K \ T_1 \ T_2
\end{aligned}$$

$$\text{Leaf} : \forall T.V \rightarrow T$$

$$\text{Leaf} = \lambda v.\lambda n.\lambda l.lv$$

Pattern matching nel caso foglia:

$$\begin{aligned}
& \text{match}_{\text{Tree}}(\text{leaf } V) \ N \ L \rightarrow_{\beta} \\
& (\lambda x.x)((\lambda v.\lambda n.\lambda l.lv)V) \ N \ L \rightarrow_{\beta} \\
& ((\lambda v.\lambda n.\lambda l.lv)V) \ N \ L \rightarrow_{\beta} \\
& (\lambda n.\lambda l.lV) \ N \ L \rightarrow_{\beta} \\
& (\lambda n.\lambda l.lV) \ N \ L \rightarrow_{\beta} \\
& (\lambda l.lV) \ L \rightarrow_{\beta} \\
& (\lambda l.lV) \ L \rightarrow_{\beta} \\
& L \ V
\end{aligned}$$

Implementazione per i naturali

Definizione del tipo associato ai numeri naturali:

$$\text{Type } N = 0 : N \mid S : N \rightarrow N$$

L'operatore di pattern matching rimane $\lambda x.x$, mentre gli elementi del tipo sono definiti come segue:

$$0 = \lambda o.\lambda s.o$$

$$S = \lambda n.\lambda o.\lambda s.sn$$

Le codifiche soprantanti dei tipi di dato algebrici seguono la codifica di Scott, che è una codifica più generale e semplice della codifica dei tipi di dato induttivi. La complessità computazionale è la medesima, tuttavia la codifica di Scott richiede di essere combinata con la ricorsione

$$\text{Ind} \subseteq \text{AlgDT}, \quad \text{Coind} \subseteq \text{AlgDT}, \quad \text{Coind} \cap \text{Ind} \neq \emptyset$$

I booleani $\in \text{Coind} \cap \text{Ind}$ in quanto non ricorsivi. I coinduttivi sono tipi infiniti, utili per processi che non terminano mai e ricevono degli input.

Richiami di teoria degli insiemi

$$A \times B = \{\langle a, b \rangle \mid a \in A, b \in B\}$$

$$A \times B = \{\langle t, x \rangle \mid t = R \wedge x \in B \vee t = L \wedge x \in A\}$$

$$\emptyset = \{\}$$

$$\mathbb{N} = \{a\}$$

Si nota che è possibile andare a definire i tipi considerandoli degli insiemi. Alcuni esempi:

•

Tipo Tree

`type Tree K V = Node: K → Tree K V → Tree K V → Tree K V | Leaf: V → Tree K V`

Utilizzando gli insiemi ottengo:

`type Tree K V = Node: K × Tree K V × Tree K V × Tree K V | Leaf: V × Tree K V`

che diventa:

`type Tree K V = K × Tree K V × Tree K V + V`

•

Tipo N Analogamente con i naturali:

$$\text{type N} = 1 + \text{N}$$

I tipi di dato algebrici prendono la forma

$$\text{type A} = \sum \prod \text{T}$$

dove T è un tipo generico. Su questo tipo di definizione di tipi, valgono tutte le operazioni algebriche classiche, che possono essere sfruttate dal compilatore per effettuare delle ottimizzazioni. Si può anche calcolare operazioni più complesse quali integrali e derivate.

Un esempio di “derivate” è dato dagli iteratori. Considero il tipo lista: $L = 1 + V \times L$ dove 1 è la lista vuota, V è la testa e L è il resto della lista. Provo a calcolare la derivata di questo tipo rispetto al tipo degli elementi V

$$\begin{aligned} \frac{dL_V}{dV} &= 0 + \frac{dV}{dV} \times L_V + \frac{dL_V}{dV} \times V \\ &= L_V + V \frac{dL_V}{dV} \end{aligned}$$

Essendo il tipo ricorsivo, anche la sua derivata è correttamente ricorsiva. Il tipo ottenuto ha la seguente forma:

$$\text{type Zipper} = \text{Future} : L_V \rightarrow \text{Zipper} \mid \text{Past} : V \rightarrow \text{Zipper} \rightarrow \text{Zipper}$$

La definizione del tipo iteratore tramite derivazione del tipo lista consente di ottenere il tipo “lista con un buco in qualche punto”, che è esattamente ciò che si può notare dalla definizione di Zipper.

Codifica dei principali costrutti tramite λ -calcolo

Codifica di variabili mutabili

```
var x = 4
f() { x = x + 1; 2 }
g(y) { x = x + y; }

main() {
  var z = f()
  g(z)
  x
}
```

Si vuole tradurre lo pseudocodice soprastante in λ -calcolo. Sapendo che ho a disposizione solamente delle funzioni, qualsiasi cosa che viene letta dovrà essere presa in input da qualche funzione e, analogamente, qualsiasi cosa che viene scritta dovrà essere output di una qualche funzione.

Dal momento che la funzione f ritorna un valore e ne modifica un altro, dovrà ritornare due valori in output: si utizza un coppia, che è codificabile in λ -calcolo.

$$f = \lambda x. \langle x + 1, 2 \rangle$$

$$g = \lambda x. \lambda y. x + y$$

$$\begin{aligned} main = \lambda x. (\\ & \text{let } \langle x', z \rangle = fx \text{ in} \\ & \text{let } x'' = gx'z \text{ in} \\ & x'' \\) \end{aligned}$$

main 4

Le variabili sono utilizzate sempre una sola volta $\rightarrow$ le funzioni sono isolabili $\rightarrow$ maggiore facilità nel testing dei programmi.

Codifica del tipo coppia

$$\text{type } A \times B = \langle _, _ \rangle : A \rightarrow B \rightarrow A \times B$$

Pattern matching per il tipo coppia:

$$\begin{aligned} & \text{match } E \text{ with} \\ & | \langle x, y \rangle \rightarrow C \end{aligned}$$

Che è letteralmente $\text{let } \langle x, y \rangle = E \text{ in } C$.

Codifica operatore let in

Sintassi di base dell'operatore **let in**: **let** $x = E$ **in** F , che si legge: “valuto E , chiamo il risultato x che può essere usato in F ”. In λ -calcolo: $(\lambda x. F)E$

Codifica operatore **for**

```
var n = 5
var tot = 0

for var i = 0; i <= n; do
    tot = tot + i
end
tot
```

Definisco l'operatore **for** tramite ricorsione.

```
let rec for = Y(λfor.λtot.λi.λn.
    match(i ≤ n)(for(tot + i)(i + 1)n) tot
)
for 0 0 5

dove Y = λf.(λx.f(xx))(λx.f(xx))
dove + e ≤ sono definiti per ricorsione sui naturali
```

Codifica operatore **while**

```
fn sum(n) {
    tot = 0
    while (n > 0) {
        tot = tot + n;
        n = n - 1;
    }
    tot
}
```

Definisco l'operatore **while** tramite ricorsione.

```
let rec while = Y(λwhile.λn.λtot.
    match(n > 0)(tot)(while(tot + n)(n - 1))

let sum = λn.while n 0

sum
```

Codifica assegnamento di campi nello heap

Si considera un esempio con alberi binari di ricerca, dove le foglie sono stringhe. Aggiorno le foglie con l'operazione **update(k, v)** dove k è la chiave intera e

v è il nuovo valore per la foglia.

Nei linguaggi funzionale deve valere l'**immutabilità**, segue che non è possibile modificare direttamente la cella di memoria $\rightarrow$ posso sfruttare questa proprietà per ridurre il costo della copia dell'intera struttura dati $\rightarrow$ vado a **modificare i puntatori** della nuova copia dell'albero affinché puntino ai dati dell'albero precedente che possono essere **riusati**.

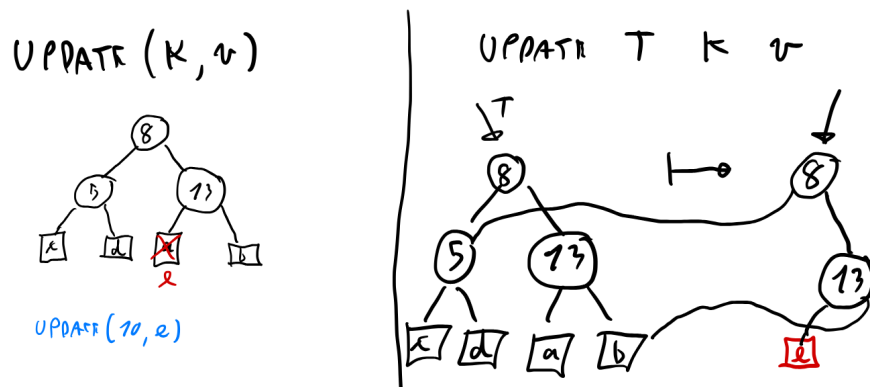


Figure 2: heap tree

Stima costo computazionale

	Funzionale	Imperativo
Spazio	$O(\log n)$	$O(1)$
Tempo	$O(\log n)$	$O(\log n)$

Ricopio solo il cammino interessato dalla modifica $\rightarrow$ costo logaritmico. La codifica in λ -calcolo non si interessa della gestione dei puntatori $\rightarrow$ classica implementazione ricorsiva.

Codifica oggetti

Esistono due tipologie di oggetti: **class-based** e **object-based**, con i primi che sono un caso particolare dei secondi. Essendo gli oggetti raccolte di dati differenti, possono essere implementati come tuple, similamente a come sono state definite le coppie in precedenza.

Si consideri il seguente frammento di pseudocodice che codifica un oggetto:

```
object:
  var x = 4
  method f = g x
```

```

method g = \y.y
method powerUp =
  x = x + 1
  g = \y.y+y

```

Il codice in λ -calcolo ha il seguente aspetto:

```

⟨4,
  (λself.self.g self self.x),
  (λself.λy.⟨self,y⟩),
  (λself.⟨self.x + 1, self.f, λself.λy.⟨self,y + y⟩, self.powerUp⟩)
⟩

```

L'oggetto consiste quindi in una tupla, avente la seguente firma:

$$\text{type } A \times B \times C \times D = \langle _, _, _, _ \rangle : A \rightarrow B \rightarrow C \rightarrow D \rightarrow A \times B \times C \times D$$

Le operazioni di accesso ai campi sono semplicemente dei pattern matching:

$$.g = \lambda r. \text{match } r \text{ with } \langle x, f, g, pu \rangle \implies g$$

Operatori di controllo

Si tratta di un operatore che cambia il classico flusso di controllo regolato dallo stack. Alcuni esempi sono: **goto**, **break**, **continue**, **catch**, **throw**, **abort**, **yield**. Il cambiamento del controllo è sia spaziale che temporale. Non è ben precisato quale sarà il destino del flusso iniziale che il controllo avrebbe dovuto seguire, potrebbe essere eliminato o ripreso successivamente $\rightarrow$ varia in base all'operatore.

Questa tipologia di operatori viene implementata a basso livello come una serie di manipolatori dello stack.

Altri operatori di controllo sono i cosiddetti "interrupt self-inflicted", che non sono altro che le syscall al sistema operativo.

Implementare questa tipologia in operatori in λ -calcolo è complesso, in quanto si tratta di un linguaggio puramente funzionale, in cui non esistono gli operatori di controllo primitivi.

Codifica di un operatore di controllo: eccezioni La codifica di un operatore di controllo significa avere la possibilità di aggiungere la disgiunzione nella firma di una funzione in λ -calcolo. Tuttavia ciò non è realizzabile in λ -calcolo, di conseguenza si ricorre alla logica per cercare una soluzione.

$$\begin{aligned}
C &\implies A \vee B \equiv \\
C &\implies \neg(\neg A \wedge \neg B) \\
C &\implies ((A \implies \perp) \wedge (B \implies \perp)) \implies \perp \\
C &\implies (A \implies \perp) \wedge (B \implies \perp) \implies \perp \\
&\quad \text{da cui segue} \\
C &\implies (A \implies \perp) \implies (B \implies \perp) \implies \perp
\end{aligned}$$

Quest'ultima formula può essere codificata in λ -calcolo.

Dinamica del λ -calcolo

Per studiare la dinamica del λ -calcolo introduciamo i **sistemi astratti di riscrittura (ARS)**.

Un ARS è una coppia $(A, \rightarrow)$:

- A è un insieme non vuoto di stati
- $\rightarrow$ è detto **passo di riscrittura** ed è una relazione su A .

Il λ -calcolo è un esempio di $\text{ARS} \rightarrow (\Lambda, \rightarrow_\beta)$.

Definizioni

- s è una **forma normale** sse $s \nrightarrow$.
- s ha **forma normale** sse $\exists s' \text{ tc } s \rightarrow^* s' \text{ e } s' \text{ è una forma normale}$.
Significa che s può convergere e viene detto **debolmente normalizzante**.
- s è **fortemente normalizzante** sse $\nexists (s_i)_{i \in \mathbb{N}} \quad \forall i. s_i \rightarrow s_{i+1}$.
- s è **deterministico** sse $\forall s_1, s_2. s_1 \leftarrow s \rightarrow s_2 \implies s_1 = s_2$.

Considerazioni (sulle definizioni)

1. fortemente normalizzante $\implies$ debolmente normalizzante
2. s deterministico $\nRightarrow$ s ha forma normale.
3. se $\rightarrow$ non ha stati normalizzanti **non è interessante**. $\forall s. s$ è deterministico, ma divergente (ovvero non debolmente normalizzante).
4. se s non è deterministico, è possibile avere **forme normali diverse** $\rightarrow$ non è cosa buona $\rightarrow$ stesso input, ma possibili output differenti.
5. se s ha forma normale s' , può capitare che $s \rightarrow s'' \nrightarrow s'$
6. s potrebbe essere **debolmente normalizzante** e ammettere un **cammino divergente** $\rightarrow$ non è **fortemente normalizzante**.

I punti 4 e 5 **non** possono accadere in λ -calcolo. Il punto 6 invece può verificarsi, ma posso sempre smettere di divergere.

Confluenza

Può essere vista come una proprietà che mitiga il non determinismo di un ARS.

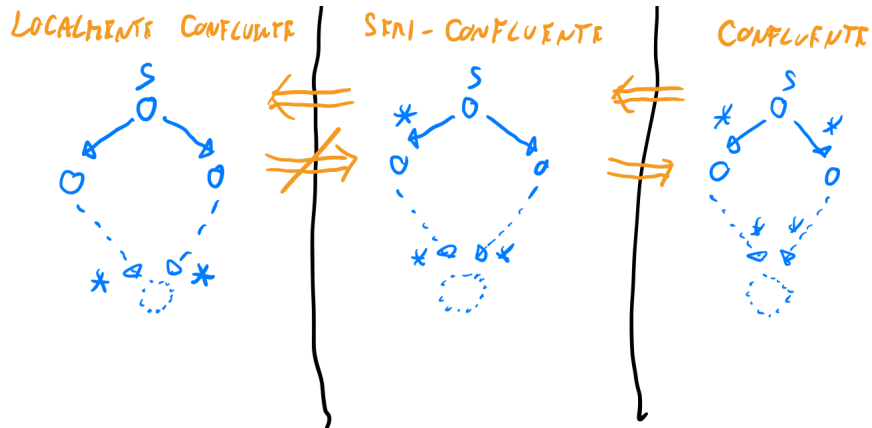


Figure 3: Confluenza

Meta-Definizione Un ARS $(A, \rightarrow)$ ha la proprietà P quando ogni $s \in A$ gode di P .

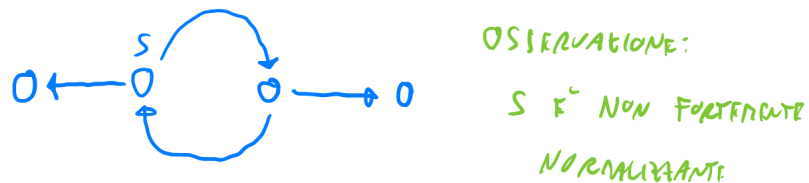


Figure 4: Locale non implica semi

Confluenza locale $\nRightarrow$ **Semi-confluenza** Sull'osservazione: localmente confluente + fortemente normalizzante $\Rightarrow$ semi-confluente.

Strip Lemma Dato un ARS semi-confluente, esso è confluente.

La dimostrazione procede per induzione su n che è il numero di passi di $\rightarrow$ nel secondo cammino, partendo da s nel caso confluente.

Riesco a *chiudere* il parallelogramma in quanto ho applicato la confluenza locale s , l'ipotesi mi dice che è semi-confluente $\Rightarrow$ localmente confluente.

Teorema: confluenza $\Rightarrow$ **unicità delle forme normali** **Idea:** se raggiugo due forme normali, queste devono coincidere: per confluenza devo potermi

Per GLI ARS SENI CONFLUENTE $\Rightarrow$ CONFLUENTE STRIP LEMMA



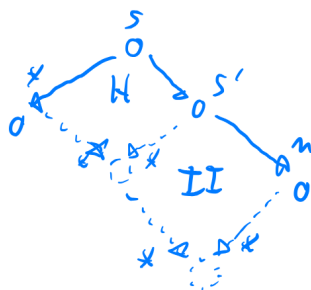
Din: VADO PER INDUZIONE SU n

Caso BASE:



QED

Caso INDUTTIVO:



II = IPOTESI
INDUTTIVA

Figure 5: Strip Lemma

TEOREMA (SU ARS): CONFLUENTE $\Rightarrow$ UNICITA' DELLE FORME NORMALI

Din:

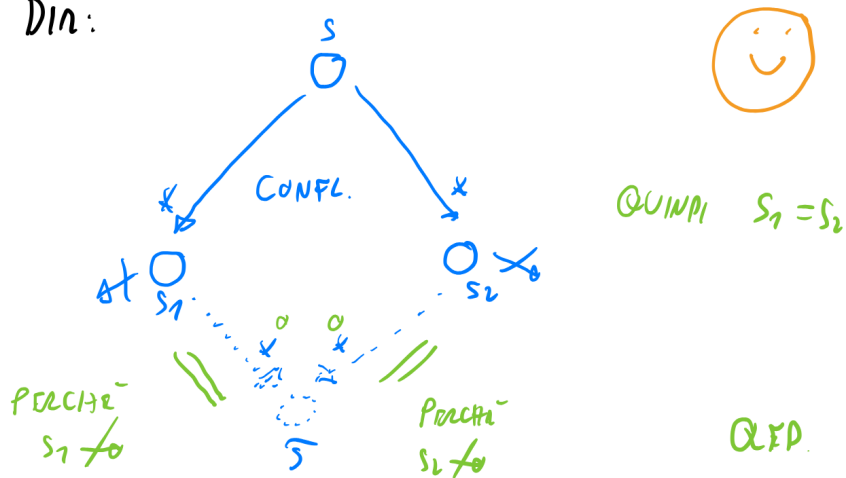
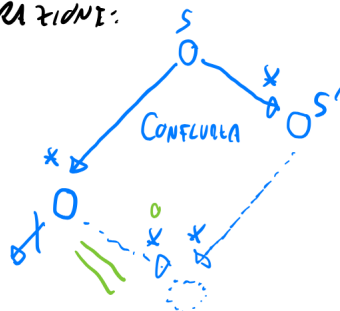


Figure 6: Unicità forme normali

ricongiungere dopo averle raggiunte, ma sono forme normali quindi da esse non posso muovermi $\rightarrow$ mi ricongiungo in 0 passi $\rightarrow$ coincidono.

Teorema: confluenza $\implies$ safety $\forall s. s \text{ confluyente} \wedge s \text{ ha forma normale} \implies (\forall s'. s \rightarrow^* s' \implies s' \text{ ha forma normale}).$

DIMOSTRAZIONE:



QED

Figure 7: Safety

Tip per capire: se s ha forma normale, allora ha un cammino che lo porta alla forma normale, applico confluenza a questo cammino e al cammino che da s va in s' .

Esempi in λ -calcolo

Non-determinismo in λ -calcolo L'unica causa di non-determinismo risiede nella scelta del redex da ridurre. Sono possibili varie casistiche:

1. Redex disgiunti

Si tratta di una forma di confluenza molto forte in cui in un passo divergo e in un altro richiudo.

Esempio: $x((\lambda y.y)z)((\lambda w.w)u)$

2. Redex parzialmente sovrapposti $\rightarrow$ non esiste in λ -calcolo.

3. Redex imbricati (uno dentro l'altro)

Generalmente uno dei due redex andrebbe distrutto, tuttavia, non è così in λ -calcolo.

Qui abbiamo due sottocasi:

- Redex imbricato nell'argomento della λ -astrazione
 - Il ramo di sinistra mostra un valutazione lazy (call-by-name), ciò porta alle seguenti osservazioni:

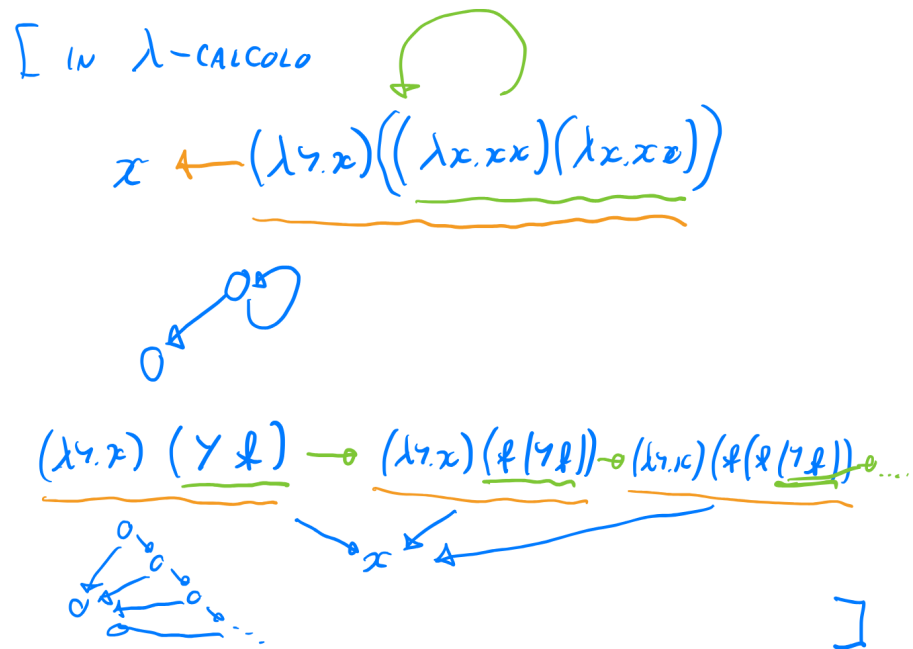


Figure 8: Safety λ -calcolo

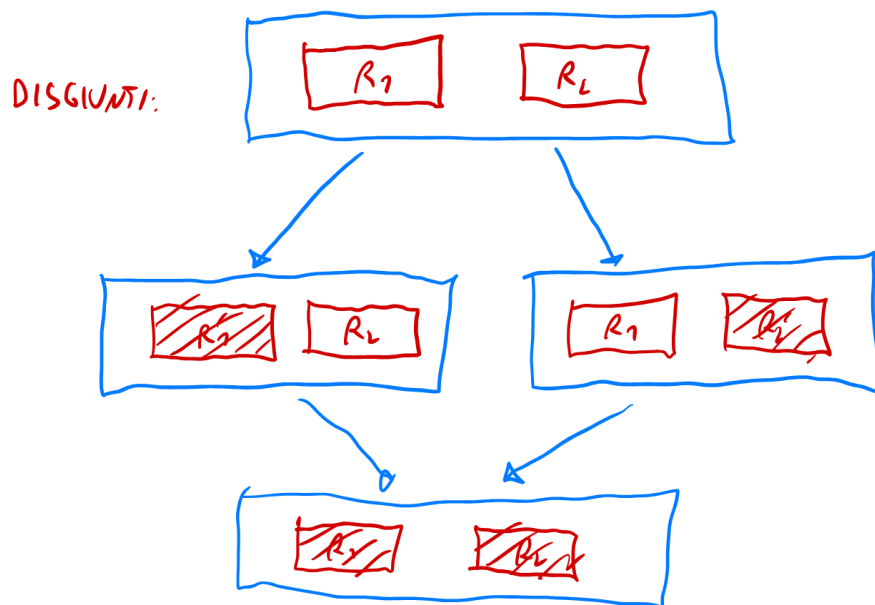


Figure 9: Redex disgiunti

ESEMPIO: SISTEMA CONCORRENTE CON DUE
PROCESSI CHE VOGLIANO ACQUISIRE UN LOCK

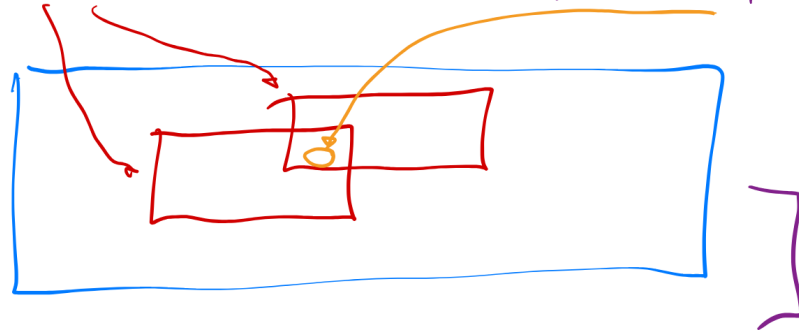


Figure 10: Redex parzialmente sovrapposti

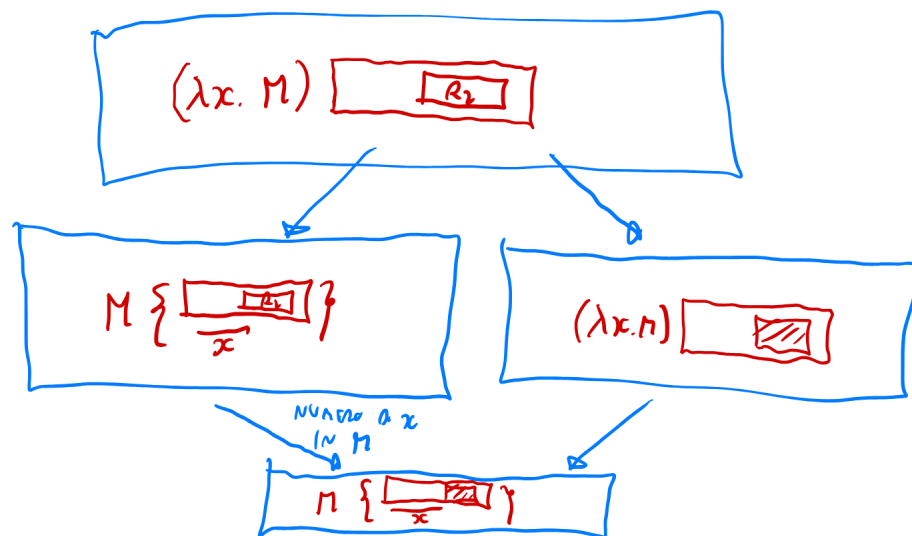


Figure 11: Imbricato argomento

1. side effect eseguiti più volte (a ogni sostituzione) → svantaggio
 2. rischio valutazione tante volte dell'argomento → svantaggio, ma ottimizzabile con memoizzazione (**call-by-mid**) → spesso l'overhead per l'ottimizzazione peggiora le performance → poco usato.
 3. se l'argomento non viene usato → non lo valuto → vantaggio
- Il ramo di destra mostra un valutazione **call-by-value**, ciò porta alle seguenti osservazioni:
1. side effect eseguiti una sola volta → vantaggio
 2. argomento valutato solo 1 volta → vantaggio
 3. se l'argomento non viene usato, lo riduco inutilmente → svantaggio

Esempio:

$$zz_\beta \leftarrow ((\lambda y.y)z)((\lambda y.y)z)_\beta \leftarrow \lambda x.xx((\lambda y.y)z) \rightarrow_\beta (\lambda x.xx)z \rightarrow_\beta zz$$

- Redex imbricato nel **corpo** della λ -astrazione

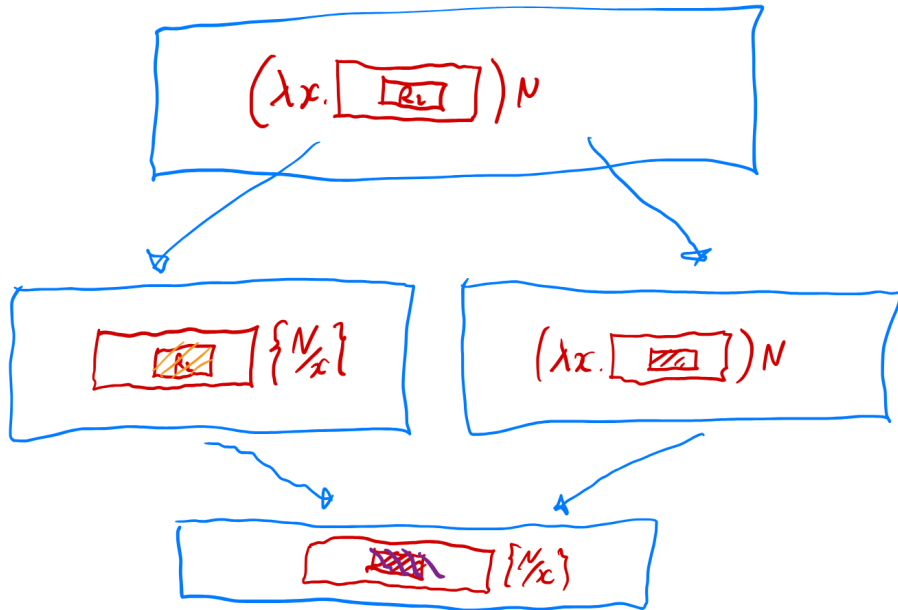


Figure 12: Imbricato corpo

- nel ramo di sinistra un redesso altera l'altro (R_2 modificato, ma non ridotto)

- nel ramo di destra R_2 viene ridotto, ma senza aver effettuato la sostituzione
- serve dimostrare un teorema per certificare che anche in questa casistica si converge (non ovvio)

Esempio:

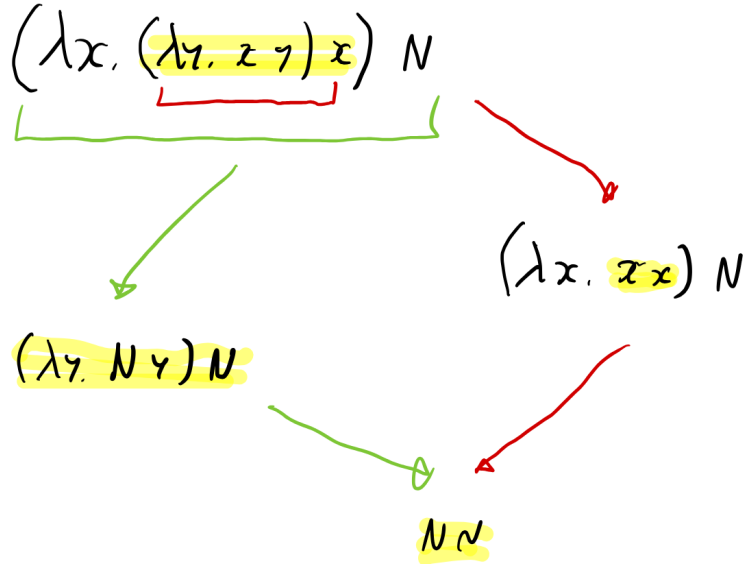


Figure 13: Esempio imbricato corpo

Dalla colorazione nell'esempio emerge l'idea della dimostrazione $\rightarrow$ **tenere traccia** dei residui ottenuti dal redesso iniziale nell'arco della dimostrazione

Teorema per dimostrare ultimo caso confluenza λ -calcolo (confluenza locale)

$$((\lambda x. M) N) \{u/y\} \rightarrow_{\beta} M \{N/x\} \{u/y\}$$

Dimostrazione

Ci sono due casistiche possibili:

- $x = y$:

Dal momento che $((\lambda x. M) N) \{u/y\} = (\lambda x. M) \{u/y\} N \{u/y\}$:

$$(\lambda x. M) \{u/x\} N \{u/x\} = (\lambda x. M) N \{u/x\} \rightarrow_{\beta} M \left\{ \frac{N \{u/x\}}{x} \right\} = M \{N/x\} \{u/x\}$$

L'ultima uguaglianza richiede un lemma per essere dimostrata. Nell'espressione $M\{\frac{N\{u/x\}}{x}\}$, le due x non sono la stessa cosa $\rightarrow$ quella sopra è **globale** e libera solo in N , mentre quella sotto compare libera in M .

- $x \neq y$:

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

$$\begin{aligned} & (\lambda x.M)N\{u/y\} \\ &= ((\lambda x.M)N)\{u/y\} \\ &= (\lambda x.M)\{u/y\}N\{u/y\} \\ &= (\lambda x.M\{u/y\})N\{u/y\} \end{aligned}$$

L'ultima uguaglianza è possibile sotto l'ipotesi $x \neq y$ e $x \notin \text{FV}(u)$.

$$\begin{aligned} & (\lambda x.M\{u/y\})N\{u/y\} \\ & \rightarrow_\beta (M\{u/y\})\{\frac{N\{u/y\}}{x}\} \\ &= M\{N/x\}\{u/y\} \end{aligned}$$

Anche quest'ultima uguaglianza è possibile sotto l'ipotesi $x \neq y$.

Lemma $(M\{N/x\}\{u/x\} = M\{\frac{N\{u/x\}}{x}\})$

Attenzione !!!

La dimostrazione come viene proposta è tecnicamente **non valida**, in quanto prima delle sostituzioni che interessano M , andrebbe introdotto un ulteriore cambio di nome di variabile. Il problema sopra nominato emerge nei casi induttivi.

Si vuole dimostrare:

$$M\{N/x\}\{u/x\} = M\{\frac{N\{u/x\}}{x}\}$$

Dimostrazione

La dimostrazione procede per induzione sulla struttura di M :

- caso x

$$x\{N/x\}\{u/x\} = N\{u/x\} = x\{\frac{N\{u/x\}}{x}\}$$

- caso y :

$$y\{N/x\}\{u/x\} = y\{u/x\} = y = y\{\frac{N\{u/x\}}{x}\}$$

- caso $\lambda z.L$:

Per la dimostrazione di questo caso si rende necessario l'impiego della seguente ipotesi induttiva:

$$L\{N/x\}\{u/x\} = L\{\frac{N\{u/x\}}{x}\}$$

$$\begin{aligned} (\lambda z.L)\{N/x\}\{u/x\} &= \\ ((\lambda w.L)\{w/z\}\{N/x\})\{u/x\} &= \quad \text{con } w \notin \text{FV}(L) \cup \text{FV}(N) \cup \text{FV}(u) \\ \lambda w.L\{w/z\}\{N/x\}\{u/x\} &= \end{aligned}$$

A questo punto posso applicare l'ipotesi induttiva su $L\{N/x\}\{u/x\}$. In questo punto si verifica la non validità della dimostrazione $\rightarrow$ dovrei modificare l'enunciato per includere $\{w/z\}$.

Nota ulteriore: la variabile w viene introdotta per evitare che z effettui catture accidentali delle variabili libere presenti in N .

$$\begin{aligned} \lambda w.L\{w/z\}\{\frac{N\{u/x\}}{x}\} &= \\ \lambda z.L\{\frac{N\{u/x\}}{x}\} & \end{aligned}$$

Quest'ultima uguaglianza è possibile grazie al fatto che w è **sufficientemente fresca**.

- caso $\lambda x.L$:

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

$$\begin{aligned} (\lambda x.L)\{N/x\}\{u/x\} &= \\ \lambda x.L &= \\ \lambda x.L\{\frac{\{u/x\}}{x}\} & \end{aligned}$$

Questo caso è molto semplice, infatti la x non è libera, e quindi **non sostituibile**.

- caso MP :

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

Per la dimostrazione di questo caso si rende necessario l'impiego delle seguenti ipotesi induttive:

$$M\{N/x\}\{u/x\} = M\{\frac{N\{u/x\}}{x}\}$$

$$P\{N/x\}\{u/x\} = P\{\frac{N\{u/x\}}{x}\}$$

La dimostrazione di questo consiste nell'applicazione delle due ipotesi induttive sfruttando la definizione della sostituzione sull'applicazione.

$$\begin{aligned} (MP)\{N/x\}\{u/x\} &= \\ (M\{N/x\}\{u/x\})(P\{N/x\}\{u/x\}) &= \\ (M\{\frac{N\{u/x\}}{x}\})(P\{\frac{N\{u/x\}}{x}\}) &= \\ (MP\{\frac{N\{u/x\}}{x}\}) \end{aligned}$$

A questo punto è stata dimostrata la **confluenza locale** del λ -calcolo $\rightarrow$ serve dimostrare la **semi-confluenza (confluenza)**.

Dimostrazione semi-confluenza λ -calcolo

L'idea alla base di questa dimostrazione si basa sull'utilizzo di un **artificio sintattico** per tenere traccia dei residui del redesso contenuto del termine iniziale. Per fare ciò viene introdotto il **λ -calcolo sottolineato**:

$$\underline{t} ::= x \mid \underline{tt} \mid \lambda x. \underline{t} \mid \underline{\lambda x. t}$$

L'unica differenza è data dall'introduzione della λ -astrazione **marcata**.

Definizione β -riduzione marcata Le regole d'inferenza della β -riduzione rimangono:

$$\overline{(\lambda x. \underline{M}) \underline{N} \rightarrow_{\beta} \underline{M}\{\underline{N}/x\}}$$

$$\frac{\underline{M} \rightarrow_{\beta} \underline{M'}}{\underline{MN} \rightarrow_{\beta} \underline{M'N}}$$

$$\frac{\underline{M} \rightarrow_{\beta} \underline{M'}}{\underline{NM} \rightarrow_{\beta} \underline{NM'}}$$

$$\frac{\underline{M} \rightarrow_{\beta} \underline{M'}}{\lambda x. \underline{M} \rightarrow_{\beta} \lambda x. \underline{M'}}$$

Vengono aggiunte le seguenti regole:

$$\overline{(\lambda x. \underline{M}) \underline{N} \rightarrow_{\beta} \underline{M}\{\underline{N}/x\}}$$

$$\frac{\underline{M} \rightarrow_{\beta} \underline{M'}}{\lambda x. \underline{M} \rightarrow_{\beta} \lambda x. \underline{M'}}$$

Osservazioni

1. Dalla prima nuova regola si intuisce che la marcatura **non altera** la computazione.
2. Dalla seconda nuova regola si intuisce che la marcatura ci permette di tenere traccia dell'evoluzione del redesso d'interesse.

Definizione sostituzione marcata $\underline{M}\{\underline{N}/x\}$ è definito come:

- $x\{\underline{M}/x\} = \underline{M}$
- $y\{\underline{M}/x\} = y$
- $(t_1 t_2)\{\underline{N}/x\} = t_1\{\underline{N}/x\} t_2\{\underline{N}/x\}$
- $(\lambda x. \underline{M})\{\underline{N}/x\} = \lambda x. \underline{M} \rightarrow$ tutte le x in M sono legate, quindi non libere e non sostituibili.
- $(\lambda y. \underline{M})\{\underline{N}/x\} = \lambda z. \underline{M}\{z/y\}\{\underline{N}/x\}$ per $z \notin \text{FV}(M) \cup \text{FV}(N)$
- $(\underline{\lambda} y. \underline{M})\{\underline{N}/x\} = \underline{\lambda} z. \underline{M}\{z/y\}\{\underline{N}/x\}$ per $z \notin \text{FV}(M) \cup \text{FV}(N)$

Funzioni di rimozione della marcatura

1. **Funzione di smarcatura:** $|\cdot|: \underline{\Lambda} \rightarrow \Lambda$. Definizione:

- $|x| = x$
- $|\underline{MN}| = | \underline{M} | | \underline{N} |$
- $|\lambda x. \underline{M}| = \lambda x. | \underline{M} |$
- $|\underline{\lambda} x. \underline{M}| = \lambda x. | \underline{M} | \rightarrow$ qui avviene la smarcatura

2. **Funzione di smarcatura e riduzione** $\rightarrow$ realizza una β -riduzione in parallelo di tutti e soli i redessi marcati. Eg. $((\underline{\lambda} x. \underline{M}) \underline{N})$. $\phi: \underline{\Lambda} \rightarrow \Lambda$.

Definizione:

- $\phi(x) = x$
- $\phi(\underline{MN}) = \phi(\underline{M})\phi(\underline{N})$ se il primo termine non è un' astrazione marcata
- $\phi(\lambda x. \underline{M}) = \lambda x. \phi(\underline{M})$
- $\phi((\underline{\lambda} x. \underline{M}) \underline{N}) = \phi(\underline{M})\{\frac{\phi(\underline{N})}{x}\}$

$$|(\underline{\lambda} \ x. (\underline{\lambda} \ y. x \ y) \ z) (\lambda n. n)| =$$

$$(\lambda x. (\lambda y. x \ y) \ z) (\lambda n. n)$$

$$\varphi \left((\underline{\lambda} \ x. (\underline{\lambda} \ y. x \ y) \ z) (\lambda n. n) \right) =$$

$$\varphi \left((\underline{\lambda} \ y. x \ y) \ z \right) \left\{ \frac{\varphi(\lambda n. n)}{x} \right\} =$$

$$(x \ z) \left\{ \frac{\lambda n. n}{x} \right\} = (\lambda n. n) \ z$$

Figure 14: Esempio funzioni smarcatura

$$\begin{array}{ccc} \underline{M} & \xrightarrow{\quad \beta \quad} & \underline{N} \\ \downarrow \text{1.1} & & \downarrow \text{1.1} \\ |\underline{M}| & \xrightarrow{\quad \beta \quad} & |\underline{N}| \end{array}$$

Figure 15: Lemma 1 enunciato

Lemma 1 Questo lemma conferma che la marcatura **non** influisce sulla computazione $\rightarrow$ smarcare e poi ridurre equivale a ridurre e poi smarcare.

Dimostrazione

Per dimostrare si procede per induzione sulla struttura dell'albero di prova $|\underline{M}| \rightarrow_\beta |\underline{N}|$:

- **Caso** $\frac{(\lambda x. N_1) N_2 \rightarrow_\beta N_1 \{N_2/x\}}$

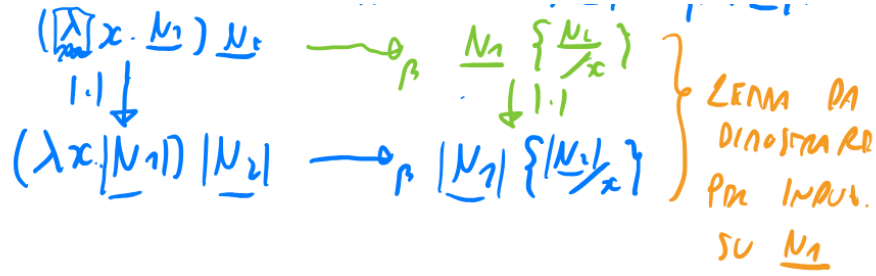


Figure 16: Lemma 1 Caso 1

Nota 1: il lemma che viene mostrato nell'immagine **non** viene dimostrato.

Nota 2: il quadratino intorno a λ indica che può trattarsi sia di λ che di $\underline{\lambda}$.

- **Caso** $\frac{M_1 \rightarrow_\beta M'_1}{M_1 M_2 \rightarrow_\beta M'_1 M_2}$

La freccia verde orizzontale è possibile in quanto H è la premessa della regola di β -riduzione $\rightarrow$ ottenuta applicando l'ipotesi induttiva. L'ipotesi induttiva è ciò che mi permette di stabilire che M'_1 nella premessa dell'albero è $|\underline{M'_1}|$

- **Caso** $\frac{M_2 \rightarrow_\beta M'_2}{M_1 M_2 \rightarrow_\beta M_1 M'_2} \rightarrow$ analogo al precedente
- **Caso** $\frac{M_2 \rightarrow_\beta M'_1}{\lambda x. M_1 \rightarrow_\beta \lambda x. M'_1} \rightarrow$ come sopra

I casi sono le 4 possibili regole del λ -calcolo classico (non sottolineato) perché stiamo trattando un passo di β -riduzione fra termini smarcati. $\rightarrow$ impossibile ci sia un'astrazione marcata.

Il ragionamento parte da questo passo di riduzione e mi chiedo dal primo termine: “quale termine posso smarcare per arrivare a quest'altro termine smarcato?”.

Lemma 2 Simile al lemma 1. Anche qui la dimostrazione avviene per induzione sull'albero di prova per $\underline{M} \rightarrow_\beta \underline{N}$. Va sottolineato che la dimostrazione sarebbe

$$\begin{array}{ccc} \text{so:} & \underline{|\mu_1|} \rightarrow_{\beta} \mu_1' & \xRightarrow{\text{1st rule 1st.}} \underline{\mu_1} \xrightarrow{H_{\beta}} \mu_1' \\ & \hline \underline{|\mu_1|} \underline{|\mu_2|} \rightarrow_{\beta} \mu_1' \underline{|\mu_2|} & & \begin{array}{ccc} \downarrow \text{1st rule 1st.} & & \downarrow \text{1st rule 1st.} \\ \underline{|\mu_1|} \rightarrow_{\beta} \underline{|\mu_1'|} & & \end{array} \\ \downarrow \text{1st rule 1st.} & & \\ \underline{|\mu_1|} \underline{|\mu_2|} & & \end{array}$$

Overall $\mu_1' = \underline{|\mu_1'|}$

Quindi $\underline{\mu_1} \xrightarrow{H} \underline{\mu_2'}$

$\downarrow \quad \downarrow$

$|\underline{\mu_1}| |\underline{\mu_2}| \longrightarrow |\underline{\mu_2'}| |\underline{\mu_1}|$

Figure 17: Lemma 1 Caso 2

una doppia induzione, in cui si aggiunge un'induzione sul numero di passi n di β -riduzione.

Enunciato

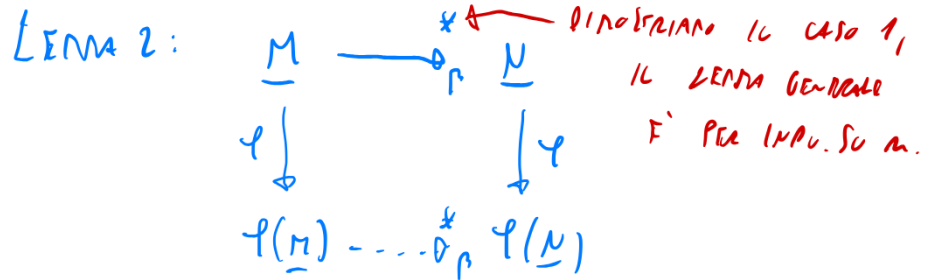


Figure 18: Lemma 2 enunciato

Dimostrazione (solo caso $n = 1$)

- Caso $\frac{M_1 \rightarrow_\beta M_1'}{\lambda x. \underline{M}_1 \rightarrow_\beta \lambda x. \underline{M}_1'}$

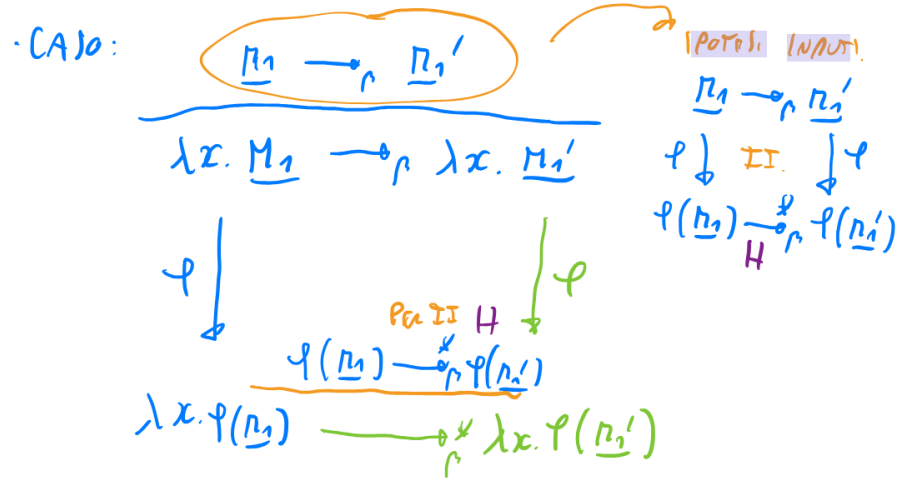


Figure 19: Caso 1 Lemma 2

- altri casi per pura ricorsione sono analoghi (sperem)
- Caso $\frac{(\lambda x. \underline{M}_1) \underline{N}_1 \rightarrow_\beta \underline{M}_1 \{ \underline{N}_1 / x \}}{(\lambda x. \underline{M}_1) \underline{N}_1 \rightarrow_\beta \underline{M}_1 \{ \underline{N}_1 / x \}}$
- Caso $\frac{(\lambda x. \underline{M}_1) \underline{N}_1 \rightarrow_\beta \underline{M}_1 \{ \underline{N}_1 / x \}}{(\lambda x. \underline{M}_1) \underline{N}_1 \rightarrow_\beta \underline{M}_1 \{ \underline{N}_1 / x \}}$

$$\begin{array}{ccc}
 (\lambda x. \underline{M}_1) \underline{N}_1 \longrightarrow_{\beta} \underline{M}_1 \left\{ \frac{\underline{N}_1}{x} \right\} & & \\
 \varphi \downarrow & & \downarrow \varphi \quad \text{Novo LEM} \\
 (\lambda x. \varphi(\underline{M}_1)) \varphi(\underline{N}_1) \longrightarrow_{\beta} \varphi(\underline{M}_1) \left\{ \frac{\varphi(\underline{N}_1)}{x} \right\} & &
 \end{array}$$

Figure 20: Lemma 2 caso 3

$$\begin{array}{ccc}
 \text{Ho } (\lambda x. \underline{M}_1) \underline{N}_1 \longrightarrow_{\beta} \underline{M}_1 \left\{ \frac{\underline{N}_1}{x} \right\} & & \\
 \varphi \downarrow & & \downarrow \varphi \quad \text{Stesso LEM} \\
 \varphi(\underline{M}_1) \left\{ \frac{\varphi(\underline{N}_1)}{x} \right\} \longrightarrow_{\beta}^{\varphi} \varphi(\underline{M}_1) \left\{ \frac{\varphi(\underline{N}_1)}{x} \right\} & &
 \end{array}$$

Figure 21: Lemma 2 caso 4

Questi due ultimi casi richiedono l'utilizzo di un lemma che va dimostrato. Una volta dimostrato questo lemma aggiuntivo, la dimostrazione del lemma 2 può considerarsi conclusa (caso $n = 1$).

NOTA !!: questo lemma aggiuntivo **non varrebbe** se si potesse creare un nuovo redesso con la sostituzione $\rightarrow$ impossibile $\rightarrow$ il lemma vale. L'unica possibilità sarebbe la seguente: $xM\{\frac{\lambda x.N}{x}\} = (\lambda x.N)M \rightarrow$ posso ritrovarmi in questa casistica solo partendo da $(\lambda x.xM)(\lambda x.N) \rightarrow$ astrazione marcata che non è la testa di un redex $\rightarrow$ **impossibile** per come avviene la marcatura e perché nel termine iniziale viene **marcata la testa di un redex**.

Lemma usato in lemma 2 Enunciato

$$\phi(\underline{M}\{\underline{N}/x\}) = \phi(\underline{M})\{\phi(\underline{N})/x\}$$

Dimostrazione La prova avviene per induzione sulla struttura ricorsiva di $\underline{M}$:

- **caso x**

$$\phi(x\{\underline{N}/x\}) = \phi(\underline{N}) = x\{\phi(\underline{N})/x\} = \phi(x)\{\phi(\underline{N})/x\}$$

- **caso y**

$$\phi(y\{\underline{N}/x\}) = \phi(y) = y = y\{\phi(\underline{N})/x\} = \phi(y)\{\phi(\underline{N})/x\}$$

- **caso $\lambda z.\underline{M}'$**

Ipotesi induttiva:

$$\phi(\underline{M}'\{\underline{N}/x\}) = \phi(\underline{M}')\{\phi(\underline{N})/x\}$$

Dimostrazione:

Assunzione per semplificare: $z \notin \text{FV}(\underline{N})$

$$\begin{aligned} \phi((\lambda z.\underline{M}')\{\underline{N}/x\}) &= \phi(\lambda z.(\underline{M}'\{\underline{N}/x\})) = \lambda z.\phi(\underline{M}'\{\underline{N}/x\}) \stackrel{\text{II}}{=} \\ \lambda z.(\phi(\underline{M}')\{\phi(\underline{N})/x\}) &= (\lambda z.\phi(\underline{M}'))\{\phi(\underline{N})/x\} = \\ \phi(\lambda z.\underline{M}')\{\phi(\underline{N})/x\} &= \end{aligned}$$

- **caso $\underline{M}_1\underline{M}_2 \rightarrow$ analogo (se $\underline{M}_1$ non è un'astrazione marcata)**

- **caso** $(\lambda z. \underline{M_1}) \underline{M_2}$

Ho delle ipotesi induttive valide sia su M_1 che M_2 .

$$\begin{aligned}
& \phi(((\lambda z. \underline{M_1}) \underline{M_2})\{N/x\}) = \\
& \phi((\lambda z. \underline{M_1}\{N/x\})(\underline{M_2}\{N/x\})) = \\
& \phi(\underline{M_1}\{N/x\})\{\frac{\phi(\underline{M_2}\{N/x\})}{z}\} = \\
& \phi(\underline{M_1})\{\phi(N)/x\}\{\frac{\phi(\underline{M_2})\{\phi(N)/x\}}{z}\} = \\
& \phi(\underline{M_1})\{\phi(\underline{M_2})/z\}\{\phi(N)/x\} =
\end{aligned}$$

Quest'ultima uguaglianza è possibile grazie a **un altro lemma** $\rightarrow M\{N/x\}\{L/y\} = M\{L/y\}\{\frac{N\{L/y\}}{x}\}$. Questo lemma vale solo se $x \notin \text{FV}(L)$, che in questo caso è vero in quanto $z \notin \text{FV}(\phi(N))$, dal momento che z è **fresca**.

$$\begin{aligned}
& \phi(\underline{M_1})\{\phi(\underline{M_2})/z\}\{\phi(N)/x\} = \\
& \phi((\lambda z. \underline{M_1}) \underline{M_2})\{\phi(N)/x\}
\end{aligned}$$

Lemma 3 Enunciato

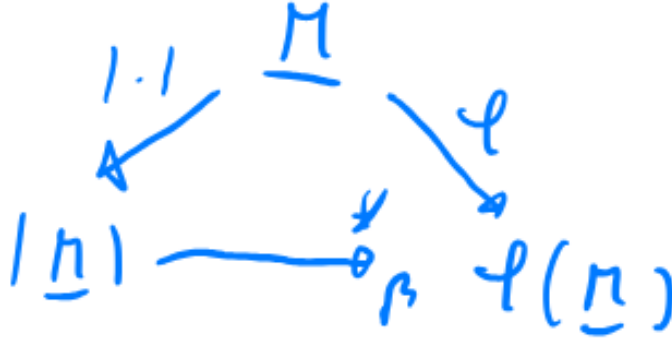


Figure 22: Lemma 3 enunciato

Dimostrazione

Per induzione su M :

- **caso** x

$$|x| = x \rightarrow_{\beta}^0 x = \phi(x)$$

- **caso** $\lambda x.\underline{N}$

Per ipotesi induttiva:

$$| \underline{N} | \rightarrow_{\beta}^* \phi(\underline{N})$$

$$| \lambda x.\underline{N} | = \lambda x. | \underline{N} | \rightarrow_{\beta}^* \lambda x.\phi(\underline{N}) = \phi(\lambda x.\underline{N})$$

Ciò che accade è l'applicazione più volte (più passi) della regola

$$\frac{M \rightarrow_{\beta} M'}{\lambda x.M \rightarrow_{\beta} \lambda x.M'}:$$

$$\frac{M_1 \rightarrow_{\beta} M_2 \rightarrow_{\beta} \cdots \rightarrow_{\beta} M_n}{\lambda x.M_1 \rightarrow_{\beta} \lambda x.M_2 \rightarrow_{\beta} \cdots \rightarrow_{\beta} \lambda x.M_n}$$

- **caso** $N_1 N_2$ (con N_1 che non sia astrazione marcata)

Ipotesi induttive:

$$| \underline{N}_i | \rightarrow_{\beta}^* \phi(\underline{N}_i) \quad \forall i \in \{1, 2\}$$

Essendo che $| \underline{N}_1 \underline{N}_2 | = | \underline{N}_1 | | \underline{N}_2 |$ e $\phi(\underline{N}_1 \underline{N}_2) = \phi(\underline{N}_1)\phi(\underline{N}_2)$, applicando le ipotesi induttive si ottiene:

$$| \underline{N}_1 \underline{N}_2 | = | \underline{N}_1 | | \underline{N}_2 | \rightarrow_{\beta}^* \phi(\underline{N}_1)\phi(\underline{N}_2) = \phi(\underline{N}_1 \underline{N}_2)$$

- **caso** $(\lambda x.\underline{N}_1)\underline{N}_2$:

Ipotesi induttive:

$$| \underline{N}_i | \rightarrow_{\beta}^* \phi(\underline{N}_i) \quad \forall i \in \{1, 2\}$$

$$\begin{aligned} & | (\lambda x.\underline{N}_1)\underline{N}_2 | \\ &= (\lambda x. | \underline{N}_1 |) | \underline{N}_2 | \\ &\rightarrow_{\beta}^* (\lambda x.\phi(\underline{N}_1))\phi(\underline{N}_2) \\ &\rightarrow_{\beta} \phi(\underline{N}_1)\{\phi(\underline{N}_2)/x\} \\ &= \phi((\lambda x.\underline{N}_1)\underline{N}_2) \end{aligned}$$

Nota: L'ultima uguaglianza vale per definizione di ϕ .

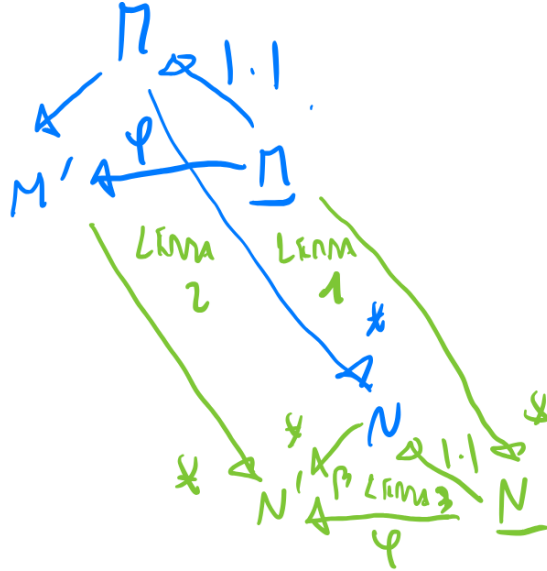


Figure 23: Teorema di semi-confluenza

Teorema di semi-confluenza per il λ -calcolo

Idea per ricordare:

- lemma 1 $\rightarrow$ permette di chiudere il *parallelogramma* con $|\cdot|$
- lemma 2 $\rightarrow$ permette di chiudere il *parallelogramma* con ϕ
- lemma 3 $\rightarrow$ permette di chiudere il *triangolo* con ϕ e $|\cdot|$

$\underline{M}$ ha come **unico** redex marcato quello che voglio ridurre in un solo passo nell'enunciato.

Dalle proprietà dei programmi ai sistemi di tipo

Viene fissato un formalismo di calcolo Turing completo (non importa quale):

- $Prog$ è l'insieme dei programmi codificabili nel formalismo
- Una proprietà di programmi P è un sottoinsieme di $Prog$. Le proprietà possono riguardare sia il comportamento (**dinamica**) che la **scrittura** del programma.
- una proprietà di programmi P è banale sse $P = \forall P = Prog$
- $p \approx q$ (p e q hanno la **stessa estensione**) sse $\forall i. (p(i) \uparrow \iff q(i) \uparrow) \wedge (p(i) \downarrow \iff q(i) \downarrow)$. Significa che i due programmi calcolano le stesse cose.
- Una proprietà P è **estensionale** sse $\forall p, q$ tc $p \approx q \implies (p \in P \iff q \in P)$
- Una proprietà è **intensionale** sse non è estensionale

- Una proprietà P è **decidibile** sse $\exists p \in Prog \text{ tc } \forall q \in Prog. ((q \in P \iff p(q) \Downarrow \text{True}) \wedge (q \notin P \iff p(q) \Downarrow \text{False}))$

Teorema di Rice

Ogni proprietà **decidibile ed estensionale** è **banale**.

Le proprietà estensionali e non decidibili sono (purtroppo) quelle più interessanti, ma non possono essere studiate $\rightarrow$ si studia un'approssimazione P' di P che sia decidibile.

Approssimazioni

Esistono 3 tipi di approssimazioni:

- **Da dentro** $\rightarrow Q$ approssima P da dentro sse $Q \subset P$. Si indica con $P^<$
- **Da fuori** $\rightarrow Q$ approssima P da fuori sse $Q \supset P$. Si indica con $P^>$
- **Miste** $\rightarrow$ non sono né di un tipo né dell'altro

Osservazione: le approssimazioni miste sono inutili

- se $p \in P^<$ allora so che $p \in P$, non so nulla se $p \notin P^<$
- se $p \notin P^>$ allora so che $p \notin P$, non so nulla se $p \in P^>$
- con il terzo tipo non ottengo nessuna informazione utile

Si vuole utilizzare **tecniche di approssimazione** per creare approssimazioni **decidibili** di proprietà **non decidibili**.

Interpretazione Astratta

Si tratta di un esempio di tecnica di approssimazione dell'esecuzione del codice. Approssima un programma utilizzando un numero **finito** di stati.

Ogni tipo di dato viene associato a un'informazione parziale $\rightarrow$ dal momento che gli stati sono finiti $\rightarrow$ i tipi vengono approssimati con dei tipi con **domini finiti**.

Un esempio di una possibile approssimazione dei gli interi $\mathbb{Z}$ potrebbe essere la seguente:

$$\overline{\mathbb{Z}} = \{pos, zero, neg\}$$

In seguito devo definire la versione approssimata delle operazioni. Esempio con $+$: $pos + pos = pos$, $neg + neg = neg$, $pos + zero = pos$, $pos + neg = \perp$.

L'ultimo esempio corrisponde a un caso in cui al risposta dell'approssimazione non fornisce alcuna informazione utile.

L'immagine riporta il **reticolo** ottenibile dal dominio del tipo approssimato $\rightarrow$ si tratta di una struttura algebrica in cui andando verso l'alto decresce il contenuto informativo del valore.

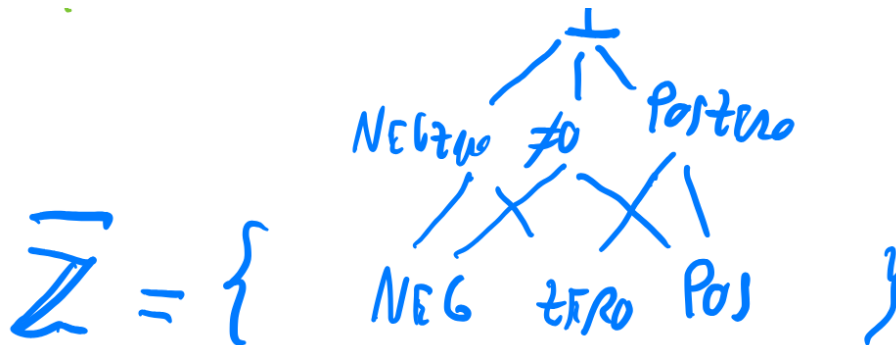


Figure 24: Reticolo AI

In seguito si estende l'astrazione anche ai **costrutti** del linguaggio. Esempio:

```
if x > 0:
    return 1
else:
    return 0
```

diventa

```
if x > zero then
    return pos
else
    return zero
```

$x = pos \mapsto pos$

$x = negpos \mapsto poszero$

Il secondo caso ritorna *poszero* perché è l'insieme dei due rami dato che non è possibile decidere.

Pro e Contro Pro:

- approssimazioni ottime
- molto utile per trovare bug in codice molto grandi e complessi

Contro:

- richiede un processo manuale complesso
- non è modulare $\rightarrow$ richiede l'analisi di tutto il codice
- costo computazionale elevato $\rightarrow$ esecuzione iterativa del codice per un numero elevato di stati

L'alternativa è rappresentata dai **sistemi di tipo**.

Sistemi di tipo

Sono approssimazioni decidibili di una qualche proprietà, con l'utile caratteristica di essere modulari. Possono essere sia da dentro che da fuori. Può anche variare il loro livello di precisione, infatti. Aumento precisione $\rightarrow$ diminuzione livello di *aiuto* (annotazioni ulteriori) del programmatore.

Si vuole verificare se una determinata unità gode della proprietà per cui il sistema di tipo è stato realizzato, come determino questa cosa sapendo che le sotto unità godono della proprietà.

Tipi: sono òa più piccola informazione che l'unità ha la proprietà.

Il passaggio dalle sotto-unità alle unità viene detto **controllo** di tipo. Quando si vuole realizzazione un sistema di tipi si procede nel seguente modo:

1. Scelta proprietà indecidibile da approssimare
2. Design e realizzazione del sistema di tipi
3. Dimostrazione di un teorema che verifichi la correttezza del sistema di tipi

Sistema di tipi semplici Presentazione alla Curry, in cui i termini sono privi di informazioni di tipo.

$$t := x \mid \lambda x.t \mid tt$$

Grammatica per i tipi:

$$T ::= \alpha \mid T \rightarrow T$$

- α è un'annotazione generica per un tipo
- $\rightarrow$ associa a destra: $A \rightarrow B \rightarrow C$ diventa $A \rightarrow (B \rightarrow C)$

Occorre realizzare la mappatura tra variabili e tipi, ciò avviene tramite un **contesto** che ha la seguente grammatica:

$$\Gamma ::= \epsilon \mid \Gamma, x : T$$

Si suppone che non esistano due entry $(x : T_1), (x : T_2)$ nel contesto. Infine viene definito il **giudizio di tipaggio** tramite un sistema d'inferenza che modella la relazione ternaria: $\Gamma \vdash t : T$

$$\frac{(x : T) \in \Gamma}{\Gamma \vdash x : T} \quad \frac{\Gamma \vdash M : T_1 \rightarrow T_2 \quad \Gamma \vdash N : T_1}{\Gamma \vdash M N : T_2} \quad \frac{\Gamma, x : T_1 \vdash M : T_2}{\Gamma \vdash \lambda x.M : T_1 \rightarrow T_2}$$

Si osserva che la terza regola **non** è un algoritmo $\rightarrow$ viene introdotto T_1 non sapendo cosa esso sia effettivamente.

$$\begin{array}{c}
\frac{(f: \alpha \multimap \beta) \in \mathcal{F} : \alpha \multimap \beta, x: \alpha}{f: \alpha \multimap \beta, x: \alpha \vdash f: \alpha \multimap \beta} \quad \frac{(x: \alpha) \in \mathcal{F} : \alpha \multimap \beta, x: \alpha}{f: \alpha \multimap \beta, x: \alpha \vdash x: \alpha} \\
\hline
f: \alpha \multimap \beta, x: \alpha \vdash f x: \beta \\
\hline
f: \alpha \multimap \beta \vdash \lambda x. f x: \alpha \multimap \beta \\
\hline
\vdash \lambda f. \lambda x. f x: (\alpha \multimap \beta) \multimap \alpha \multimap \beta
\end{array}$$

Figure 25: Derivazione di tipo

Esempio di tipaggio per $\lambda f. \lambda x. f x$

L'obiettivo è andare a compilare i tipi. $\rightarrow$ si parte con degli spazi vuoti, quando si arriva in alto, verranno compilati i primi tipi, che verranno poi riscritti fino ad arrivare alla fine.

Non tutte le espressioni in λ -calcolo possono essere tipate $\rightarrow$ esempio $\lambda x. x x$

Church vs Curry

- Church propone l'approccio basato su **type checking** $\rightarrow$ programmatore annota esplicitamente i tipi e type system esegue controlli sulle annotazioni.
- Curry propone l'approccio basato su **type inference** $\rightarrow$ il compilatore annota i tipi da solo.

Il secondo approccio ha due contro:

- tipi non annotati $\rightarrow$ codice meno documentato (risolvibile con un buon IDE)
- in alcuni casi capita che la spiegazione dell'errore di tipaggio sia errata.

Logica proposizionale minimale

$$F ::= \alpha \mid F \rightarrow F$$

- $\rightarrow$ implicazione logica associativa a destra.

Ipotesi (contesto):

$$\Gamma ::= \epsilon \mid \Gamma, F$$

$\Gamma \vdash F$ relazione binaria descritta tramite sistema d'inferenza:

$\circ = \circ \rightarrow$

NESSUN TIPO $\circ$
 SODDISFA
 L'EQUAZIONE

$$\frac{\frac{(x:\rightarrow) \in x:\rightarrow}{x:\rightarrow \vdash x:\circ \rightarrow}}{x:\rightarrow \vdash x x:}$$

$$\frac{\frac{(x:\rightarrow) \in x:\rightarrow}{x:\rightarrow \vdash x:\circ \rightarrow}}{x:\rightarrow \vdash x x:}$$

$$\vdash \lambda x. x x:$$

Figure 26: Derivazione fallita

$$\frac{F \in \Gamma}{\Gamma \vdash x : T} \quad \frac{\Gamma \vdash F_1 \rightarrow F_2 \quad \Gamma \vdash F_1}{\Gamma \vdash F_2} \quad \frac{\Gamma, F_1 \vdash F_2}{\Gamma \vdash F_1 \rightarrow F_2}$$

La seconda è l'**eliminazione** dell'implicazione. La terza è l'**introduzione** dell'implicazione.

Isomorfismo di Curry-Howard-Kolmogorov

$$\frac{(x:T) \in \Gamma}{\Gamma \vdash x : T} \quad \frac{\Gamma \vdash M : T_1 \rightarrow T_2 \quad \Gamma \vdash N : T_1}{\Gamma \vdash M N : T_2} \quad \frac{\Gamma, x : T_1 \vdash M : T_2}{\Gamma \vdash \lambda x. M : T_1 \rightarrow T_2}$$

$$\frac{F \in \Gamma}{\Gamma \vdash x : T} \quad \frac{\Gamma \vdash F_1 \rightarrow F_2 \quad \Gamma \vdash F_1}{\Gamma \vdash F_2} \quad \frac{\Gamma, F_1 \vdash F_2}{\Gamma \vdash F_1 \rightarrow F_2}$$

Si nota che i due sistemi d'inferenza sono pressoché **identici**.

λ -calcolo	Logica
Tipi	Formule
Termini	Prova (albero)
variabili di tipo	Variabili proposizionali
Costruttori di tipo ($\times$, unioni disgiunte, AlgDT)	Connettivi logici

λ -calcolo	Logica
Costrutti del linguaggio	Regole di introduzione e di eliminazione
Variabili libere	Ipotesi globali
Variabili legate	Ipotesi locali
Type checking (Γ, t, T sono gli input)	Proof checking
Type inhabitation (Γ, T sono input, t output)	Ricerca della prova
Normalizzazione	Normalizzazione / Cut elimination
Redex	Introduzione + Eliminazione
Tipi di dato algebrici	Forme normali disgiunte

Aumentando la complessità della logica in esame, aggiungo nuovi costrutti al mio linguaggio, ed ho la garanzia che essi siano dei buoni costrutti **grazie all'isomorfismo**.

I λ -termini equivalgono alla prova, in quanto osservando bene si nota che sono essi stessi che determinano la struttura dell'albero di derivazione.

Subject reduction (TH)

Proprietà buona che un buon sistemi di tipi dovrebbe avere. $\rightarrow$ la riduzione preserva il tipaggio.

$$\forall \Gamma, t_1, t_2, T. \Gamma \vdash t_1 : T \wedge t_1 \rightarrow^* t_2 \implies \Gamma \vdash t_2 : T$$

Per l'isomorfismo, lato logica, questa proprietà sembra tradursi nella possibilità di riscrivere la dimostrazione in modo diverso.

Congiunzione

$$F ::= \dots \mid F \wedge F$$

$$T ::= \dots \mid T \times T$$

$$t ::= \dots \mid \langle t, t \rangle \mid t.1$$

$$\langle t_1, t_2 \rangle.1 \rightarrow t_1$$

$$\langle t_1, t_2 \rangle.2 \rightarrow t_2$$

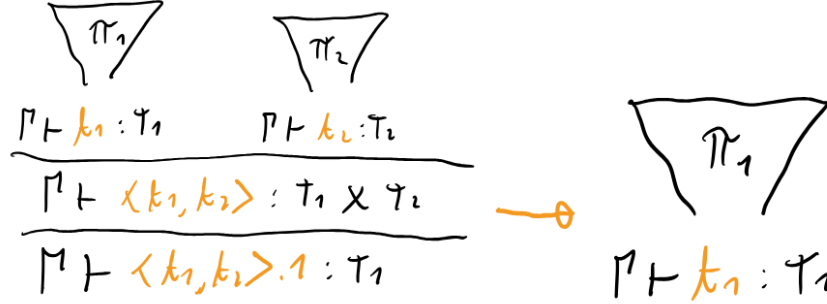


Figure 27: SR

Congiunzione (eliminazione alternativa)

$$F ::= \dots \mid F \wedge F$$

$$T ::= \dots \mid T \times T$$

$$t ::= \dots \mid \langle t, t \rangle \mid (x, y) = x; M$$

$$\langle t_1, t_2 \rangle; M \rightarrow M\{t_1/x, t_2/y\}$$

La seconda dimostrazione è ottenuta in modo analogo a come viene ottenuta la seconda dimostrazione dell'implicazione.

Definizione come tipo algebrico

$$\text{type } A \times B = \langle, \rangle : A \rightarrow B \rightarrow A \times B$$

Implicazione

$$(\lambda x.M)N \rightarrow_{\beta} M\{N/x\}$$

Riguardo $\pi_1\{N/x\}$ non ho assunzioni, in quanto non so nulla su N . Le x che erano in M nella prima dimostrazione, sicuramente comparivano nelle foglie per come è stato definito il sistema di inferenza $\rightarrow$ ora in quelle foglie devo dimostrare N invece di x . Ciò avviene *incollando* le dimostrazioni di N della prima dimostrazione, come mostrato nella figura.

$$\begin{array}{c}
\begin{array}{c} \triangle \pi_1 \\ \Gamma \vdash k_1 : T_1 \end{array} \quad \begin{array}{c} \triangle \pi_2 \\ \Gamma \vdash k_2 : T_2 \end{array} \\
\hline
\Gamma \vdash \langle k_1, k_2 \rangle : T_1 \times T_2 \quad \begin{array}{c} \triangle \pi_3 \\ \Gamma, x:T_1, y:T_2 \vdash n : T_3 \end{array} \\
\hline
\Gamma \vdash (x, y) = \langle k_1, k_2 \rangle, n : T_3 \quad \text{--- } \theta
\end{array}$$

$$\begin{array}{c}
\begin{array}{c} \triangle \pi_1 \quad \dots \quad \triangle \pi_i \\ \hline \triangle \pi_j \{k_1/x, k_i/y\} \end{array} \\
\Gamma \vdash n \{k_1/x, k_i/y\} : T_3
\end{array}$$

Figure 28: SR

$$\begin{array}{c}
\begin{array}{c} \triangle \pi_1 \\ \Gamma, x:T_1 \vdash n : T_2 \end{array} \quad \begin{array}{c} \triangle \pi_2 \\ \Gamma \vdash N : T_1 \end{array} \\
\hline
\Gamma \vdash \lambda x. n : T_1 \rightarrow T_2 \quad \Gamma \vdash N : T_1 \\
\hline
\Gamma \vdash (\lambda x. n) N : T_2 \quad \text{--- } \theta
\end{array}$$

$$\begin{array}{c}
\begin{array}{c} \triangle \pi_1 \quad \dots \quad \triangle \pi_n \\ \hline \triangle \pi_1 \{N/x\} \end{array} \\
\Gamma \vdash n \{N/x\} : T_2
\end{array}$$

Figure 29: SR

Disgiunzione

$$F ::= \dots \mid F \vee F$$

$$T ::= \dots \mid T + T$$

$$t ::= \dots \mid \iota_1 t \mid \iota_2 t \mid$$

$$\text{match } t \text{ with } \mid \iota_1 x \Rightarrow M \mid \iota_2 y \Rightarrow N$$

$$\frac{\Gamma \vdash a : F_1}{\Gamma \vdash \iota_1 a : F_1 \vee F_2} \vee_{i_1} \quad \frac{\Gamma \vdash b : F_2}{\Gamma \vdash \iota_2 b : F_1 \vee F_2} \vee_{i_2}$$

$$\frac{\Gamma \vdash d : F_1 \vee F_2 \quad \Gamma, x : F_1 \vdash M : F_3 \quad \Gamma, y : F_2 \vdash N : F_3}{\Gamma \vdash \text{match } d \text{ with } \begin{array}{l} \iota_1 x \Rightarrow M \\ \iota_2 y \Rightarrow N \end{array} : F_3} \vee$$

Figure 30: SR

Definizione come tipo algebrico

$$\text{type } A + B =$$

$$\mid \iota_1 : A \rightarrow A + B$$

$$\mid \iota_2 : B \rightarrow A + B$$

Teorema (forme normali disgiunte)

Ogni formula F della logica proposizionale classica è **logicamente equivalente** a una **disgiunzione di congiunzioni** >

$$F \equiv \vee_i \wedge_j G_{ij} \quad \text{dove } G_{ij} = A \text{ o } G_{ij} = \neg A \text{ con } A \text{ variabile proposizionale}$$

Le formule G_{ij} prendono il nome di **forme normali disgiunte**

Esempio

$$\begin{aligned}
& (\neg A \implies B) \wedge (C \vee \neg D) \\
& \equiv (\neg \neg A \vee B) \wedge (C \vee \neg D) \\
& \stackrel{distr.}{\equiv} (\neg \neg A \wedge C) \vee (\neg \neg A \wedge \neg D) \vee (B \wedge C) \vee (B \wedge \neg D) \\
& \equiv A \wedge C \vee A \wedge \neg D \vee \neg B \wedge C \vee \neg B \wedge \neg D
\end{aligned}$$

Questo teorema è un risultato molto importante della logica, grazie all'isomorfismo di Curry-Howard-Kolmogorov è possibile trovare un corrispondente lato informatica, che sono i **tipi di dato algebrici**.

Intuitivamente le disgiunzioni di congiunzioni sono somme disgiunte di tuple, che non sono altro che i tipi di dato algebrici.

Top

$$F ::= \dots \mid \top$$

$$T ::= \dots \mid 1$$

$$t ::= \dots \mid * \mid () = t; M$$

Il tipo che si ottiene è il tipo **unit**. Spesso nei linguaggi, sia il tipo, che l'operatore associato sono denotati con $()$.

$$\frac{\Gamma \vdash x : T_i \quad \frac{\Gamma \vdash k : T \quad \Gamma \vdash n : T_1}{\Gamma \vdash : () = k ; n : T_1}}{\Gamma \vdash x : T_i}$$

Figure 31: SR

Definizione come tipo algebrico

$$\text{type } \mathbb{K} = * : \mathbb{K}$$

Bottom

$$F ::= \dots \mid \perp$$

$$T ::= \dots \mid$$

$$t ::= \dots \mid \text{abort } t \mid \text{raise } e \mid \text{assert} \mid \text{match } t \text{ with}$$

$$\frac{\Gamma \vdash t : \perp}{\Gamma \vdash \text{abort } t : \top} \quad \text{!e}$$

Figure 32: SR

Definizione come tipo algebrico

type =

Tipo privo di abitanti.

Logica proposizionale al secondo ordine

$F ::= \dots \mid \forall \alpha. F \mid \exists \alpha. F$ con α variabile proposizionale

$T ::= \dots \mid \forall \alpha. T \mid \exists \alpha. T$

$t ::= \dots \mid \boxed{t} \mid \boxed{x} = M; t$

La SOL è sintatticamente simile alla FOL. La differenza risiede in ciò su cui agiscono i quantificatori $\rightarrow$ nella FOL si quantifica sugli oggetti, nella SOL invece si quantifica sulle formule. $\rightarrow$ più espressiva.

I termini vengono definiti alla Curry $\rightarrow$ aggiungo il tipaggio, ma il linguaggio **non** cambia $\rightarrow$ posso applicare ad uno stesso programma due sistemi di tipi differenti, a seconda del grado di approssimazione che desidero.

Regole di introduzione e eliminazione per $\forall$

$$\frac{\Gamma \vdash t : F\{\beta/\alpha\}}{\Gamma \vdash t : \forall \alpha. F} \quad \forall_i \quad \beta \notin \text{FV}(M)$$

Per dimostrare il $\forall$ mi riduco a dimostrare F con una generica β (non ho assunzioni al riguardo) al posto di α in F .

$$\frac{\Gamma \vdash t : \forall \alpha. F}{\Gamma \vdash t : F\{G/\alpha\}} \quad \forall_e$$

Se la formula F vale per tutte le α , allora continuerà a valere anche per una generica formula G al posto di α .

Applico l'isomorfismo Applicando l'isomorfismo di Curry-Howard-Kolmogorov alle regole per $\forall$ si ottiene il **polimorfismo**. Grazie all'isomorfismo è possibile tipare il termine $\lambda x.xx$ che non era tipabile in lambda calcolo tipato semplice.

$\lambda x.xx$ Non è tipabile nel λ -calcolo tipato semplice

$\{ \forall d. d \rightarrow d \}$

$$\begin{array}{c}
 \frac{(x : \forall d. d \rightarrow d) \in x : \forall d. d \rightarrow d}{x : \forall d. d \rightarrow d \vdash x : \forall d. d \rightarrow d} \\
 \frac{x : \forall d. d \rightarrow d \vdash x : (\forall d. d \rightarrow d) \rightarrow \forall d. d \rightarrow d \quad x : \forall d. d \rightarrow d \vdash x : \forall d. d \rightarrow d}{x : \forall d. d \rightarrow d \vdash xx : \forall d. d \rightarrow d} \Rightarrow_e \\
 \frac{\vdash \lambda x.xx : (\forall d. d \rightarrow d) \rightarrow \forall d. d \rightarrow d}{\vdash \lambda x.xx : (\forall d. d \rightarrow d) \rightarrow \forall d. d \rightarrow d} \Rightarrow_\lambda
 \end{array}$$

Figure 33: $\lambda x.xx$ tipabile in System F

Tuttavia il termine $(\lambda x.xx)(\lambda x.xx)$ **non è tipabile** con le regole a nostra disposizione. Come succedeva andando a tipare $\lambda x.xx$ in λ -calcolo tipato semplice ho un mismatch $\rightarrow$ da una parte il mio termine deve avere tipo *con la freccia*, dall'altro no.

System F e ADT

La codifica dei tipi vista e poi *cancellata* a inizio corso usava proprio il system F.

Esempi:

$$\mathbb{B} = \forall \alpha. \alpha \rightarrow \alpha \rightarrow \alpha$$

$$\mathbb{N} = \forall \alpha. \alpha \rightarrow (\mathbb{N} \rightarrow \alpha) \rightarrow \alpha$$

$$= \forall \alpha. \alpha$$

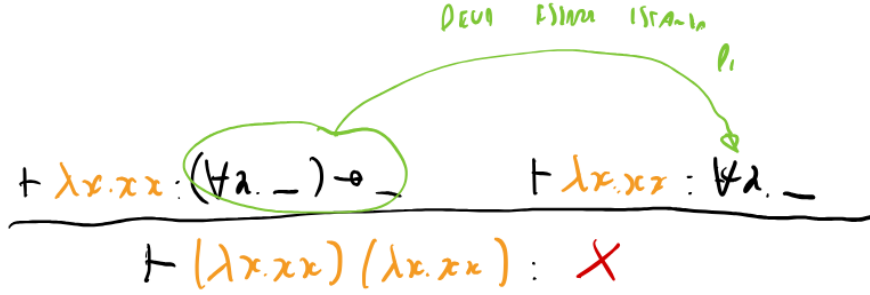


Figure 34: Termine non tipabile in system F

Il tipo vuoto è il tipo delle funzioni con output polimorfo e input assente $\rightarrow$ non ho casi per il pattern matching.

Esempio particolare

$\forall \alpha. \mathbb{B} \rightarrow \alpha$

Funzione che prende in input un booleano e ritorna un valore polimorfo $\rightarrow$ funzione che diverge poiché per rispettare la dichiarazione di tipo deve richiamare se stessa.

Il tipaggio polimorfico permette di avere una chiara descrizione di cosa fa la funzione.

Regole di introduzione e eliminazione per $\exists$

A differenza dell'universale, aggiungendo le regole esistenziali, sono obbligato a modificare il linguaggio.

$$\frac{\Gamma \vdash t : F\{G/\alpha\}}{\Gamma \vdash \boxed{t} : \exists \alpha. F} \quad \exists_i$$

Per dimostrare $\exists$ mi riduco a dimostrare F con una qualche G al posto di α in F .

$$\frac{\Gamma \vdash m : \exists \alpha. F \quad \Gamma, x : F\{\beta/\alpha\} \vdash t : G}{\Gamma \vdash \boxed{x} = m; t : G} \quad \exists_e \quad \beta \notin \text{FV}(\Gamma) \cup \text{FV}(G)$$

Se ho m che ha tipo astratto $\exists \alpha. F$ e un codice t di tipo G , sapendo che x è un'implementazione concreta di m , che non dipende dalle ipotesi o dal codice, allora sarò sicuro che x è ottenibile solo dal tipo di dato astratto.

L'operatore $\boxed{\cdot}$ rimuove informazioni su quale sia effettivamente il tipo F .

Applico l'isomorfismo Applicando l'isomorfismo di Curry-Howard-Kolmogorov aggiungo i **tipi di dato astratti**.

Esempio: Stack

$$\begin{array}{l} \exists.\text{Stack}. \\ \text{Stack} \\ \times (\alpha \rightarrow \text{Stack} \rightarrow \text{Stack}) \quad \text{push} \\ \times (\alpha \rightarrow (\alpha \times \text{Stack}) + 1) \quad \text{pop} \end{array}$$

Applicando a regola di eliminazione, trovo una implementazione concreta del tipo di dato astratto.

$$\begin{array}{l} \boxed{x} = c; \\ \langle \text{empty}, \text{push}, \text{pop} \rangle = x; \end{array}$$

Ora posso usare empty, push, pop nel mio codice.

Proprietà approssimata dal sistema di tipi.

System F **non** tipa $(\lambda x.xx)(\lambda x.xx)$ che è il più piccolo termine divergente $\rightarrow$ operatore di punto fisso applicato all'identità.

La proprietà approssimata è la **normalizzazione forte**.

Teorema di normalizzazione forte

$$\forall \Gamma, t, T. \Gamma \vdash T : t \implies t \text{ è fortemente normalizzante}$$

Il λ -calcolo tipato è fortemente normalizzante.

Corollario

λ -calcolo tipato **non** è Turing completo.

Infatti $(\lambda x.xx)(\lambda x.xx)$ è istanza di $Y = (\lambda x.f(xx))(\lambda x.f(xx))$ che **non** è tipabile.

Rendere λ -calcolo tipato Turing completo Si aggiunge una regola assiomatica al type system per tipare $Y \rightarrow Y : \forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \alpha$.

Dimostrazione del teorema di normalizzazione forte

Primo tentativo errato

La dimostrazione procede per induzione strutturale su t :

- caso x :

Se ho una variabile allora l'unico modo per ottenerla è tramite la regola seguente:

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T}$$

Non si possono più applicare regole per estendere verso l'alto la derivazione quindi x è fortemente normalizzante.

- caso $\lambda x.M$:

Se mi ritrovo in questa casistica allora significa che la seguente regola è stata applicata:

$$\frac{\Gamma, x : T_1 \vdash M : T_2}{\Gamma \vdash \lambda x.M : T_1 \rightarrow T_2} \rightarrow_i$$

La dimostrazione si conclude con successo dal momento che per ipotesi induttiva M è fortemente normalizzante.

- caso MN :

Significa che la seguente regola è stata utilizzata:

$$\frac{\Gamma \vdash M : T_1 \rightarrow T_2 \quad \Gamma \vdash N : T_1}{\Gamma \vdash MN : T_2} \rightarrow_e$$

Per ipotesi induttiva M e N sono entrambi fortemente normalizzanti. Ci si accorge subito che non si riesce a mostrare che MN è normalizzante. Questo si verifica perché la normalizzazione (sia forte che debole) **non è modulare**.

Controesempio $M = N = \lambda x.xx$

Riducibilità

Visto l'impossibilità di dimostrare la normalizzazione forte, emerge che il sistema di tipi approssima un'altra proprietà: la **riducibilità** $\rightarrow$ una versione *indebolita* della normalizzazione che ha la peculiarità di essere modulare.

Occorre dare una definizione più stringente di questa nuova proprietà $\rightarrow$ la combinazione degli elementi deve rispettare il tipaggio degli elementi:

Riducibile di Tipo $T =$ fortemente normalizzante $\wedge$ tipato con tipo $T \wedge$ combinandolo con altri riducibili di tipo compatibile rimane riducibile

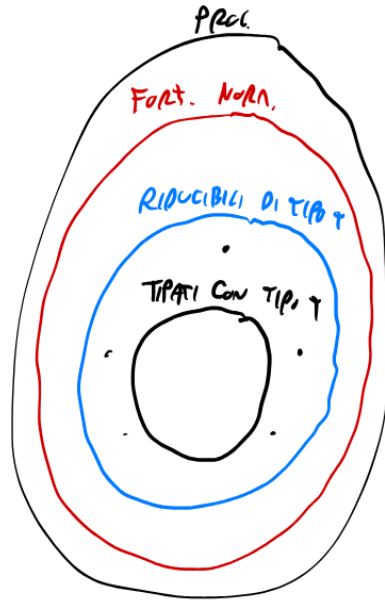


Figure 35: Proprietà

Definizione: riducibile di tipo T (λ -calcolo tipato semplice)

$$T ::= \alpha \mid T \rightarrow T$$

$$\text{RID}_{\alpha} = \{t \mid \Gamma \vdash t : \alpha \wedge t \in SN\}$$

$$\text{RID}_{T_1 \rightarrow T_2} = \{t \mid \Gamma \vdash t : T_1 \rightarrow T_2 \wedge t \in SN \wedge \forall N \in \text{RID}_{T_1}. tN \in \text{RID}_{T_2}\}$$

Con questa nuova nozione della proprietà è possibile portare a termine la dimostrazione precedentemente fallita.

Fondamenti Logici dell'Informatica (Modulo 2)

Verifica di sistemi reattivi - logiche temporali

L'utilizzo di testing per verificare la correttezza di sistemi concorrenti e/o distribuiti:

- numero di test finiti
- difficile fare test esaustivi
- il testing mostra la presenza di bug, **ma non l'assenza.**

Sistemi critici richiedono soluzioni differenti dal testing classico.

Model Checking

- utilizzo di logiche per specificare il comportamento di un programma $\rightarrow$ realizzazione di tool che verificano se il **modello** soddisfa le **specifiche**.
- sistemi complessi si possono evolvere nel tempo $\rightarrow$ servono logiche in grado di codificare dinamiche temporali come “mai”, “prima o poi”, “successivo”.

Nota: spesso modelli complessi vengono modellati come sottomodelli separati, in questo caso serve unirli nuovamente in un unico modello, in modo da poter ragionare correttamente su di esso.

Esempi di proprietà desiderate

- **Safety** $\rightarrow$ non succede **mai** nulla di male
- **Progress** $\rightarrow$ il sistema può **sempre** evolvere
- **Liveness** $\rightarrow$ **prima o poi** capita qualcosa di buono

Il tempo gioca un ruolo centrale.

Strutture di Kripke

Una **Struttura di Kripke** è quadrupla (S, s_0, R, L) dove:

- S è un insieme di **stati**
- s_0 è lo **stato iniziale**
- $R \subset S \times S$ è la **relazione di transizione**
- $L : S \rightarrow \phi(AP)$ è una **funzione di etichettatura** che indica quali proposizioni atomiche sono vere in ogni stato.

Con S finito per motivi di decidibilità. Notazione: $s \rightarrow t$ per indicare $(s, t) \in R$.

Cammini

Sequenza finita di stati che inizia da s_0 tc $\forall i. s_i \rightarrow s_{i+1}$.

Ogni cammino è una possibile soluzione del sistema, si considerano solo cammini **infiniti** dal momento che la relazione di transizione è **totale** $\rightarrow$ ogni stato ha sempre una possibile transizione in uscita.

Suffisso Per n-esimo suffisso di un cammino π si intende il cammino al quale sono stati rimossi i primi n passi e si indica con π^n .

Nota: $\pi^0 = \pi$

Logica Temporale Lineare - LTL

$$F, G ::= P \mid F \wedge G \mid F \vee G \mid \neg F \mid \mathbf{X}F \mid \mathbf{F}F \mid \mathbf{G}F \mid FUG$$

- P è una proposizione atomica
- **FXGU** sono **connettivi temporali**, in particolare:

- **X** → neXt → prossimo
- **G** → Globally → sempre
- **F** → Future → prima o poi
- **U** → Until → finché

Semantica

Il **cammino** π soddisfa la formula F nella struttura di Kripke $\mathcal{M}$

$$\mathcal{M}, \pi \models F$$

$$\begin{aligned} (S, s, R, L), t, \pi \models P &\iff P \in L(t) \\ \mathcal{M}, \pi \models F \wedge G &\iff \mathcal{M}, \pi \models F \text{ e } \mathcal{M}, \pi \models G \\ \mathcal{M}, \pi \models F \vee G &\iff \mathcal{M}, \pi \models F \text{ oppure } \mathcal{M}, \pi \models G \\ \mathcal{M}, \pi \models \neg F &\iff \text{non è vero che } \mathcal{M}, \pi \models F \\ \mathcal{M}, \pi \models \mathbf{X}F &\iff \mathcal{M}, \pi^1 \models F \\ \mathcal{M}, \pi \models \mathbf{G}F &\iff \mathcal{M}, \pi^i \models F \text{ per ogni } i \in \mathbb{N} \\ \mathcal{M}, \pi \models \mathbf{F}F &\iff \mathcal{M}, \pi^i \models F \text{ per almeno un } i \in \mathbb{N} \\ \mathcal{M}, \pi \models F\mathbf{U}G &\iff \text{se esiste } i \in \mathbb{N} \text{ tale che } \mathcal{M}, \pi^i \models G \\ &\quad \text{e } \mathcal{M}, \pi^j \models F \text{ per ogni } 0 \leq j < i \end{aligned}$$

La struttura di Kripke $\mathcal{M} = (S, s, R, L)$ soddisfa la formula F

$$\mathcal{M} \models F \iff \mathcal{M}, \pi \models F \text{ per ogni cammino } \pi \text{ che inizia da } s$$

LTL usa implicitamente una quantificazione universale sui cammini → per verifica di condizioni esistenziali utilizzo la **negazione**.

Modellazione e validazione di un sistema tramite LTL

Prima versione del modello Si vuole modellare un sistema in cui Alice e Bob possono fare uscire i rispettivi gatto e cane nel giardino condiviso, senza che i due animali si incontrino → utilizzo di due luci (una per attore) che vengono accese dai due attori per orchestrare la presenza dei rispettivi animali nel giardino.

Formulazione proprietà tramite LTL

- Alice e Bob riescono *sempre* ad accendere la propria luce, se è spenta.
- Alice e Bob riescono *sempre* a spegnere la propria luce, se è accesa.



Figure 36: Esemplio LTL 2

$$\mathbf{G}(S_B \implies \mathbf{F}(A_B)) \quad \mathbf{G}(S_A \implies \mathbf{F}(A_A))$$

$$\mathbf{G}(A_B \implies \mathbf{F}(S_B)) \quad \mathbf{G}(A_A \implies \mathbf{F}(S_A))$$

dove:

- $S_A \equiv$ Luce di Alice spenta
- $S_B \equiv$ Luce di Bob spenta
- $A_A \equiv$ Luce di Alice accesa
- $A_B \equiv$ Luce di Bob accesa

Queste proprietà **non** sono soddisfatte in quanto Alice o Bob potrebbero ripetere in loop il seguente cammino:

accensione luce $\rightarrow$ animale esce $\rightarrow$ animale rientra

La proprietà di **progress** non è rispettata in questo modello.

- se Alice accende la propria luce, prima o poi il gatto esce
- se Bob accende la propria luce, prima o poi il cane esce

$$\mathbf{G}(A_A \implies \mathbf{F}G)$$

$$\mathbf{G}(A_B \implies \mathbf{F}D)$$

dove:

- $G \equiv$ Gatto di Alice
- $D \equiv$ Cane di Bob

Anche queste proprietà non vengono rispettate $\rightarrow$ Alice e Bob accendono insieme.
 $\rightarrow$ deadlock.

Questo modello **non** soddisfa la proprietà di liveness.

Miglioramento del modello Il problema principale è dato dalla simmetria nei cammini che i due attori possono intraprendere all'interno del modello $\rightarrow$ occorre introdurre un comportamento differente per un attore affinché uno abbia priorità rispetto l'altro.

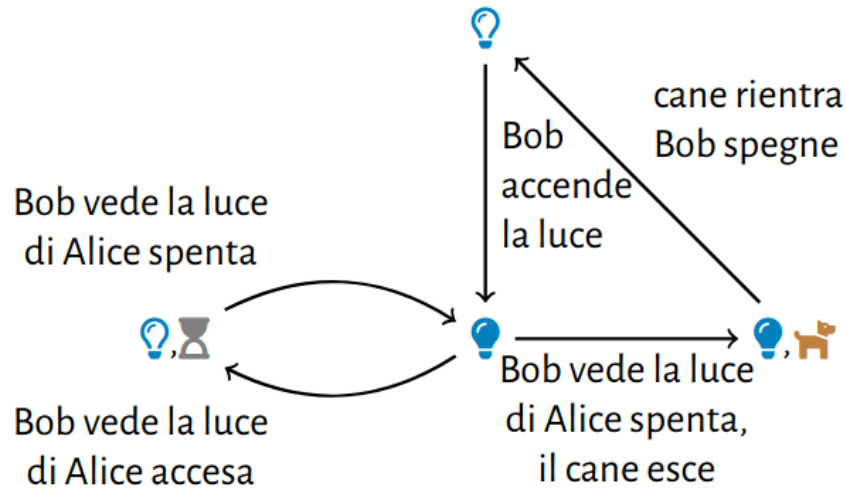


Figure 37: Esempio LTL 1

Introducendo il comportamento soprastante Alice ha priorità su Bob.

Rispetto alla prima versione del modello si può osservare un miglioramento $\rightarrow$ la **liveness** è garantita per Alice. Le altre proprietà rimangono non soddisfatte.

LTL fissa le scelte degli attori nel sistema per esaminare un cammino specifico per volta. $\rightarrow$ affiancata da altre logiche che consentono di esaminare i cammini universalmente.

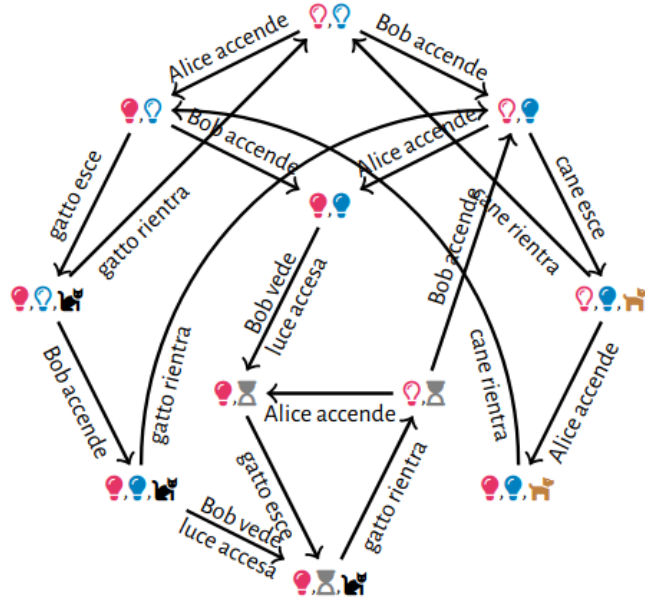


Figure 38: Esempio LTL 3

Proprietà di non alternanza Sistemi che proteggono la mutua esclusione tramite alternanza fissata sono banali e introducono assenza di **garanzia di progresso**.

C'è un cammino lungo il quale Alice e Bob non si alternano ?

Che diventa:

è falso che in ogni cammino Alice e Bob si alternano sempre.

Formula LTL:

$$\mathbf{G}(G \implies \mathbf{F}(\neg G \wedge \neg GUD))$$

“Se il gatto è in cortile allora dopo un po' il gatto non è più in cortile e non lo è più fino a quando compare il cane”.

Computational Tree Logic - CTL

Limitazioni di LTL

- quantificazione universale su tutti i cammini
- non tutte le proprietà che verificano l'esistenza di un cammino con certe caratteristiche possono essere espresse

- proprietà che mescolano quantificazione universale ed esistenziale sui cammini **non sono esprimibili in LTL**
- **logiche branching time** $\rightarrow$ introducono operatori di quantificazione sui cammini **A** e **E**, rispettivamente universale ed esistenziale.
- “branching time” = ragionare sull’insieme dei cammini possibili, piuttosto che su ciascuno preso individualmente come fa LTL.

Sintassi CTL

$$F, G ::= P \mid F \wedge G \mid F \vee G \mid \neg F \mid \Omega \mathbf{X} F \mid \Omega \mathbf{F} F \mid \Omega \mathbf{G} F \mid \Omega[F \mathbf{U} G]$$

$$\Omega ::= \mathbf{A} \mid \mathbf{E}$$

dove:

- **A** = “per ogni cammino”
- **E** = “esiste un cammino”

Nota!! Il quantificatore precede **immediatamente** il connettivo temporale.

Semantica CTL

Lo stato s soddisfa la formula F nella struttura di Kripke $\mathcal{M}$

$$\mathcal{M}, s \models F$$

$$(S, s, R, L), t \models P \iff P \in L(t)$$

$$\mathcal{M}, s \models F \wedge G \iff \mathcal{M}, s \models F \text{ e } \mathcal{M}, s \models G$$

$$\mathcal{M}, s \models F \vee G \iff \mathcal{M}, s \models F \text{ oppure } \mathcal{M}, s \models G$$

$$\mathcal{M}, s \models \neg F \iff \text{non è vero che } \mathcal{M}, s \models F$$

$$\mathcal{M}, s \models \mathbf{A} \mathbf{X} F \iff \forall s' \text{ tc } s \rightarrow s'. \mathcal{M}, s' \models F$$

$$\mathcal{M}, s \models \mathbf{E} \mathbf{X} F \iff \exists s' \text{ tc } s \rightarrow s' \text{ e } \mathcal{M}, s' \models F$$

$$\mathcal{M}, s_0 \models \mathbf{A} \mathbf{G} F \iff \forall \text{ cammino } s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \text{ si ha } \mathcal{M}, s_i \models F \quad \forall i \in \mathbb{N}$$

$$\mathcal{M}, s_0 \models \mathbf{E} \mathbf{G} F \iff \exists \text{ cammino } s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \text{ tc } \mathcal{M}, s_i \models F \quad \forall i \in \mathbb{N}$$

$$\mathcal{M}, s_0 \models \mathbf{A} \mathbf{F} F \iff \forall \text{ cammino } s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \exists i \in \mathbb{N} \text{ tc } \mathcal{M}, s_i \models F$$

$$\mathcal{M}, s_0 \models \mathbf{E} \mathbf{F} F \iff \exists \text{ cammino } s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \exists i \in \mathbb{N} \text{ tc } \mathcal{M}, s_i \models F$$

$$\mathcal{M}, s \models \mathbf{A}[F \mathbf{U} G] \iff \forall \text{ cammino } s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \exists i \in \mathbb{N} \text{ tc}$$

$$\mathcal{M}, s_i \models G \text{ e } \mathcal{M}, f_j \models F \quad \forall 0 \leq j < i$$

$$\mathcal{M}, s \models \mathbf{E}[F \mathbf{U} G] \iff \exists \text{ cammino } s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \exists i \in \mathbb{N} \text{ tc}$$

$$\mathcal{M}, s_i \models G \text{ e } \mathcal{M}, f_j \models F \quad \forall 0 \leq j < i$$

Di seguito alcune raffigurazioni che semplificano il confronto fra alcuni operatori poco intuitivi.

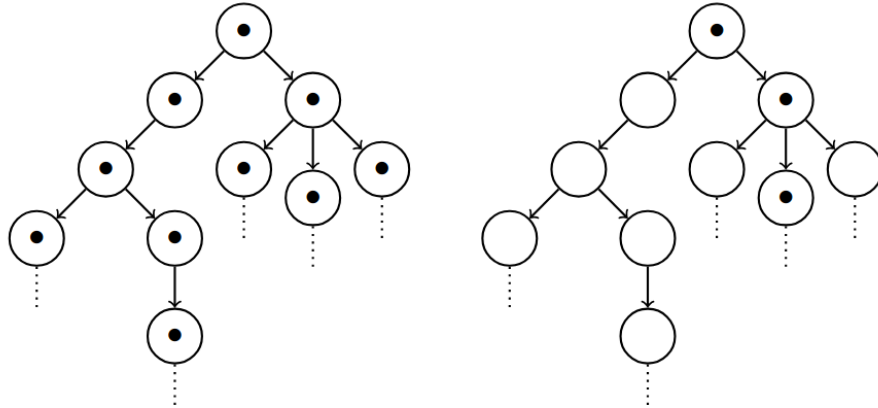


Figure 39: AG vs EG

AG vs EG

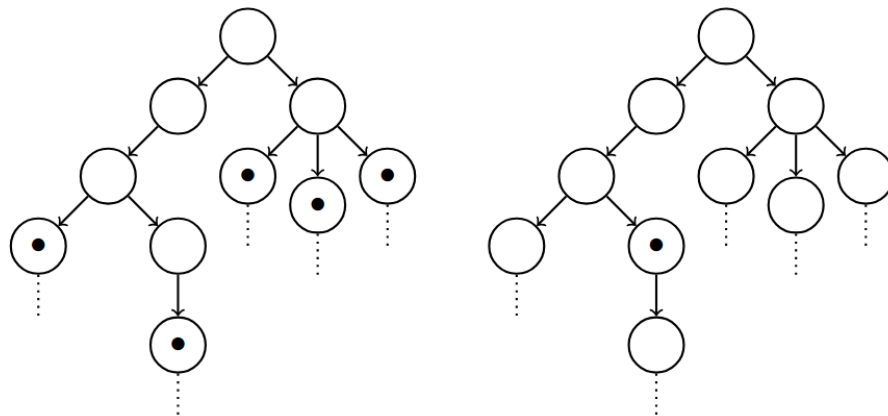


Figure 40: AF vs EF

AF vs EF

Potere espressivo di LTL e CTL

LTL e CTL sono **incomparabili** dal punto di vista del poter espressivo.

Sembra chiaro che CTL può esprimere proprietà che LTL non può, tuttavia esistono proprietà esprimibili in LTL, ma non in CTL.

Esempio di proprietà esprimibile in LTL, ma non in CTL Si consideri $F(\mathbf{G}p)$:



Figure 41: LTL vs CTL

- $F(\mathbf{G}p)$ è falsa $\rightarrow$ esiste almeno un cammino in cui non è vera (es. $q_0q_1 \dots$)
- $\mathbf{AF}(\mathbf{EG}p)$ è vera

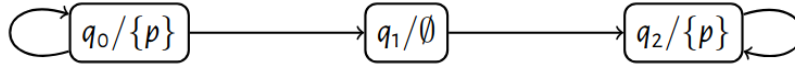


Figure 42: LTL vs CTL

- $F(\mathbf{G}p)$ è vera $\rightarrow$ ho solo due cammini possibili:
 - $q_0q_0 \dots$
 - $q_0q_0 \dots q_0q_1q_2q_2 \dots$
- $\mathbf{AF}(\mathbf{AG}p)$ è false $\rightarrow$ un cammino passa attraverso q_1 dove p non vale

Esempio di proprietà non esprimibile in LTL **Teorema:** Non esiste alcuna formula LTL equivalente a $\mathbf{AG}(\mathbf{EF}p)$

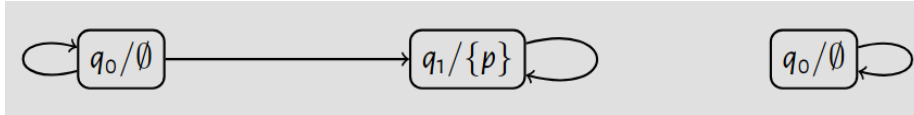


Figure 43: LTL vs CTL

Dimostrazione:

- Si suppone che esista un formula LTL F equivalente a $\mathbf{AG}(\mathbf{EF}p)$
- Il modello a sinistra soddisfa $\mathbf{AG}(\mathbf{EF}p)$
- $\mathbf{AG}(\mathbf{EF}p) \equiv F \rightarrow$ il modello a sinistra deve soddisfare F
- I cammini nel modello di destra sono cammini nel modello di sinistra
- Segue che il modello a destra soddisfa F (F vera in tutti gli stati del modello a sinistra $\rightarrow$ cammini a destra sono sottoinsieme di quelli a sinistra $\rightarrow F$ vera in tutti gli stati del modello a destra)
- Il modello a destra non soddisfa $\mathbf{AG}(\mathbf{EF}p) \rightarrow$ assurdo $\rightarrow$ cvd

Model Checker per CTL

Le regole più complesse su cui fare model checking sono quelle dalla settima in poi poiché sono possibili **infiniti cammini** (gli stati sono **finiti**).

Formulazione del problema

Formulazione classica:

Dato un modello $\mathcal{M} = (S, s_0, R, L)$ e una formula F , stabilire se $\mathcal{M} \models F$.

Formulazione alternativa:

- Dato un modello $\mathcal{M} = (S, s_0, R, L)$ e una formula F , calcolare il più grande sottoinsieme X di S tale che $\mathcal{M}, s \models F$ per ogni $s \in X$.
- $\mathcal{M} \models \mathcal{F}$ sse $s_0 \in X$

La seconda formulazione sfrutta la finitezza degli stati per dare una semantica alternativa. Il sottoinsieme di stati X della definizione, viene indicato con $[[F]]$.

Definizione di $[[F]]$ La definizione è per induzione sulla grammatica della logica.

La logica ha numero elevato di operatori, che rendono più complessa la definizione per induzione. Viene quindi scelto un **insieme adeguato di connettivi** $\rightarrow$ un sottoinsieme dei connettivi originali della logica tale che permette di riscrivere **tutte** le formule della logica usando solo quei connettivi.

Teorema L'insieme di connettivi $\{\mathbf{EX}, \mathbf{AF}, \mathbf{EU}\}$ è adeguato

Dimostrazione

- $\mathbf{AX}F \equiv \neg \mathbf{EX} \neg F$
- $\mathbf{EFF} \equiv \mathbf{E}[\top \cup F]$
- $\mathbf{EG}F \equiv \neg \mathbf{AF} \neg F$
- $\mathbf{AG}F \equiv \neg \mathbf{EF} \neg F$
- $\mathbf{A}[FUG] \equiv \neg(\mathbf{E}[\neg GU(\neg F \wedge \neg G)] \vee \mathbf{EG} \neg G)$

Definizione di $\mathbf{EX}F$ Si definisce quindi $[[\mathbf{EX}F]] = \text{pre}[[F]]$ dove pre è la **preimmagine**:

$$\text{pre}(X) \stackrel{\text{def}}{=} \{s \in S \mid \exists t \in X : R(s, t)\}$$

Intuizione: la preimmagine è l'insieme di quegli stati dai quali posso essere partito per arrivare allo stato corrente in un passo.

Definizione di $\mathbf{AFF}$ Per definire $[[\mathbf{AFF}]]$ prendiamo prima in considerazione la formulazione di ricorsiva della formula.

$$\mathbf{AFF} \equiv F \vee \mathbf{AXAFF}$$

Espandendo questa formulazione troviamo la definizione:

$$\begin{aligned} [[\mathbf{AFF}]] &= [[F \vee \mathbf{AXAFF}]] \\ &= [[F]] \cup [[\mathbf{AXAFF}]] \\ &= [[F]] \cup [[\neg \mathbf{EX} \neg \mathbf{AFF}]] \\ &= [[F]] \cup (S \setminus [[\mathbf{EX} \neg \mathbf{AFF}]])) \\ &= [[F]] \cup (S \setminus \text{pre}[[\neg \mathbf{AFF}]]) \\ &= [[F]] \cup (S \setminus \text{pre}(S \setminus \text{pre}([[\mathbf{AFF}]])))) \end{aligned}$$

Per terminare serve calcolare l'equazione $[[\mathbf{AFF}]] = [[F]] \cup (S \setminus \text{pre}(S \setminus \text{pre}([[\mathbf{AFF}]]))))$ che è un problema di **punto fisso**.

Teorema (Minimo punto fisso nel caso finito)

Si tratta di una caso particolare del **punto fisso di Kleene**.

Sia A un insieme finito e sia $\mathcal{F} : \wp(A) \rightarrow \wp(A)$ una funzione **monotona**. Allora esiste k tale che $\mathcal{F}^k()$ è il **più piccolo punto fisso** di $\mathcal{F}$, che viene denotato con $\mu\mathcal{F}$.

Il teorema mi dice che, applicando $\mathcal{F}$ per un numero finito di volte, a partire da $\emptyset$, arriverò a ottenere un insieme finito di elementi che non crescerà più. Questo insieme è il più piccolo punto fisso. Il vantaggio è che è calcolabile in modo **totalmente programmatico**.

$\wp(A) \rightarrow$ insiemi finiti di elementi

Grazie a questo teorema possiamo completare la definizione:

$$[[\mathbf{AFF}]] = \mu\mathcal{F} \quad \text{dove} \quad \mathcal{F}(X) = [[F]] \cup (S \setminus \text{pre}(S \setminus X))$$

Nota: $\mathcal{F}$ è **monotona** (quindi valie il teorema) $\rightarrow$ vene applicato un numero pari di differenze.

Definizione di $\mathbf{E}[FUG]$ Anche qui prendiamo in considerazione la formulazione ricorsiva della formula.

$$\mathbf{E}[FUG] \equiv G \vee (F \wedge \mathbf{EXE}[FUG])$$

Conti con la nuova formulazione:

$$\begin{aligned}
& [[\mathbf{E}[FUG]]] \\
& = [[G \vee (F \wedge \mathbf{EXE}[FUG])]] \\
& = [[G]] \cup [[F \wedge \mathbf{EXE}[FUG]]] \\
& = [[G]] \cup ([[F]] \cap [[\mathbf{EXE}[FUG]]]) \\
& = [[G]] \cup ([[F]] \cap \text{pre}[[\mathbf{E}[FUG]]])
\end{aligned}$$

Quindi:

$$[[\mathbf{E}[FUG]]] = \mu\mathcal{F} \quad \text{dove} \quad \mathcal{F}(X) = [[G]] \cup ([[F]] \cap \text{pre}(X))$$

Logica Lineare

Il teorema $A \rightarrow (A \rightarrow B) \rightarrow A \wedge B$

In logica classica la formula soprastante è un teorema. Dargli un'interpretazione sensata è molto semplice:

- $A = \text{ipotesi} \rightarrow n \text{ è multiplo di } 6$
- $A \rightarrow B = \text{teorema} \rightarrow \text{se } n \text{ è multiplo di } 6, \text{ allora } n \text{ non è primo}$
- $A \wedge B = \text{tesi} \rightarrow n \text{ è multiplo di } 6 \text{ e } n \text{ non è primo}$

Tuttavia quando si ragiona con delle **risorse** emergono dei problemi:

- $A = \text{stato iniziale} \rightarrow \text{ho un euro}$
- $A \rightarrow B = \text{distributore automatico} \rightarrow \text{consumo un euro e produco un caffè}$
- $A \wedge B = \text{stato finale} \rightarrow \text{ho un euro e un caffè}$

La conclusione ottenuta non va bene, non rispetta la semantica del sistema $\rightarrow$ causato dall'utilizzo multiplo delle ipotesi nella dimostrazione del teorema $\rightarrow$ utilizzo **non lineare** delle ipotesi.

Intuitivamente una risorsa può essere consumata una sola volta.

Due congiunzioni

In logica delle risorse ci sono due congiunzioni: **additiva** e **moltiplicativa**.

Congunzione additiva

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}$$

Significa che con Γ è possibile ottenere sia A che B separatamente.

Congunzione moltiplicativa

$$\frac{\Gamma \vdash A \quad \Delta \vdash B}{\Gamma, \Delta \vdash A \wedge B}$$

Weakening e Contraction

- Weakening

$$\frac{\Gamma \vdash A}{\Gamma, B \vdash A}$$

- Contraction

$$\frac{\Gamma, B, B \vdash A}{\Gamma, B \vdash A}$$

Queste regole in logica sono intuitive $\rightarrow$ le ipotesi sono verità e **riutilizzabili** quando serve. Tramite queste 2 regole in logica classica è possibile derivare una congiunzione dall'altra e viceversa $\rightarrow$ in logica tradizionale non c'è differenza tra le due.

Da congiunzione additiva a moltiplicativa

$$\frac{\frac{\Gamma \vdash A}{\Gamma, \Delta \vdash A} \text{weakening} \quad \frac{\Delta \vdash B}{\Gamma, \Delta \vdash B} \text{weakening}}{\Gamma, \Delta \vdash A \wedge B} \wedge \text{additiva}$$

Da congiunzione moltiplicativa a additiva

$$\frac{\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma, \Gamma \vdash A \wedge B} \wedge \text{moltiplicativa}}{\Gamma \vdash A \wedge B} \text{contraction}$$

Formule della logica lineare

$$A, B ::= \mathbf{1} \mid \perp \mid \top \mid \mathbf{0} \mid A \otimes B \mid A \wp B \mid A \& B \mid A \oplus B$$

	Congiunzione	Disgiunzione	Unità	Unità
moltiplicativa	$\otimes$	$\wp$	$\mathbf{1}$	$\perp$
additiva	$\&$	$\oplus$	$\top$	$\mathbf{0}$

Gergo:

- $\otimes$ = *tensore o per*
- $\wp$ = *par*
- $\&$ = *with*
- $\oplus$ = *plus più*

Nonostante il nome fuorviante *par*, il operatore che realizza il parallelismo è il tensore. Questa sintassi descrive il frammento MALL (Multiplicative, Additive Linear Logic) che è **privo di negazione** $\rightarrow$ non serve la negazione esplicita in quanto in logica lineare c'è il concetto di **dualità**.

Dualità

Se A significa “scelgo/produco”, allora $A^\perp$ significa “offro/consumo” (e viceversa).

$$\begin{aligned}
 (\mathbf{1})^\perp &= \perp \\
 (\perp)^\perp &= \mathbf{1} \\
 (\top)^\perp &= \mathbf{0} \\
 (\mathbf{0})^\perp &= \top \\
 (A \otimes B)^\perp &= A^\perp \wp B^\perp \\
 (A \wp B)^\perp &= A^\perp \otimes B^\perp \\
 (A \& B)^\perp &= A^\perp \oplus B^\perp \\
 (A \oplus B)^\perp &= A^\perp \& B^\perp
 \end{aligned}$$

La dualità è un’**involuzione** $\rightarrow A^{\perp\perp} = A$ per ogni A .

Sequenti

1. Logica Intuizionista (Modello Asimmetrico) Il sequente è:

$$A_1, \dots, A_n \vdash B$$

Che è equivalente a dire:

$$A_1 \wedge \dots \wedge A_n \vdash B$$

- **Significato:** Se ho tutte le input A (congiunzione $\wedge$), allora posso produrre B .
- A destra del simbolo $\vdash$ (turnstile) c’è **una sola formula** (B).
- **Interpretazione Informatica (Asimmetrica):**
 - Questo rappresenta una **funzione**.
 - Le A sono gli **argomenti** (input) della funzione.
 - La B è il **valore di ritorno** (output).
 - È “asimmetrico” perché puoi avere molti input ma restituisci sempre **un solo risultato**.

2. Logica Lineare Classica (Modello Simmetrico) Il sequente è:

$$A_1, \dots, A_m \vdash B_1, \dots, B_n$$

Che corrisponde a:

$$A_1 \otimes \dots \otimes A_m \vdash B_1 \wp \dots \wp B_n$$

- **Significato:** Qui i simboli cambiano. A sinistra usiamo il **prodotto tensore** ($\otimes$, una “e” forte) e a destra il **Par** ($\wp$, una “o” parallela).
- Ci sono più formule sia a sinistra che a destra ($B_1, \dots, B_n$).
- **Interpretazione Informatica (Simmetrica):**

- Non più funzioni, ma **processi** o **scambi di risorse**.
- **Risorse consumate (A)**: In logica lineare, le ipotesi non sono verità eterne, ma “gettoni” che spendi. Se usi A per ottenere qualcosa, A non c’è più.
- **Risorse prodotte (B)**: Il processo non restituisce un solo valore, ma produce un nuovo stato o un insieme di output complessi interagenti.
- È “simmetrico” perché input e output hanno la stessa struttura “multipla”.

Logica one-sided La simmetria della logica lineare consente l’utilizzo di sequenti *one sided*:

$$A_1, \dots, A_m \vdash B_1, \dots, B_n \approx \vdash A_1^\perp, \dots, A_m^\perp, B_1, \dots, B_n$$

Regole

Costanti

$$\frac{}{\vdash \mathbf{1}} \quad \frac{}{\vdash \Gamma, \top} \quad \frac{\vdash \Gamma}{\vdash \Gamma, \perp} \quad \text{nessuna regola per } \mathbf{0}$$

- $\mathbf{1}$ è l’unità moltiplicativa, **significato intuitivo** $\rightarrow$ *ho finito*
- $\perp$ è il duale di $\mathbf{1}$, **significato intuitivo** $\rightarrow$ *hai finito?* $\rightarrow$ è l’attesa di un segnale, poi continua.
- $\top$ è l’unità additiva, **significato intuitivo** $\rightarrow$ *cosa scegli tra 0 possibilità* $\rightarrow$ interpretabile come la descrizione di un programma che va in crash.
- $\mathbf{0}$ è il duale di $\top$, **significato intuitivo** $\rightarrow$ *scelgo tra 0 possibilità* $\rightarrow$ rappresenta un programma impossibile da realizzare

Connettivi moltiplicativi

$$\frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B} \quad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \wp B}$$

- $A \otimes B \rightarrow$ **significato intuitivo** $\rightarrow$ servono sia Γ che Δ per produrre A e B insieme, dato che avevo bisogno di Γ per produrre A e che avevo bisogno di Δ per produrre B . Sia A che B vengono eseguiti in un ordine scelto dall’**esterno**.
- $A \wp B \rightarrow$ il *par* fa le veci della virgola. Entambe le azioni eseguite secondo un ordine **interno**.

Connettivi additivi (contesto condiviso)

$$\frac{\vdash \Gamma, A \quad \vdash \Gamma, B}{\vdash \Gamma, A \& B} \quad \frac{\vdash \Gamma, A}{\vdash \Gamma, A \oplus B} \quad \frac{\vdash \Gamma, B}{\vdash \Gamma, A \oplus B}$$

- $A \& B \rightarrow$ **significato intuitivo** $\rightarrow$ posso ottenere A e B separatamente con le risorse in Γ , ma non posso ottenerli insieme. $\rightarrow$ **scelta esterna**, infatti il sistema dev'essere pronto per produrre entrambe le scelte con le stesse risorse (non sa a priori quale sarà la scelta). Si tratta di un'**offerta** fra due scelte verso l'esterno.
- $A \oplus B \rightarrow$ **significato intuitivo** $\rightarrow$ scelta **interna** di chi produrre fra A e B .

Altre regole

$$\frac{}{\vdash A, A^\perp} \text{ assioma} \quad \frac{\vdash \Gamma, A \quad \vdash \Delta, A^\perp}{\vdash \Gamma, \Delta} \text{ taglio (o cut)}$$

- **assioma** $\rightarrow$ **significato intuitivo** $\rightarrow$ posso produrre solo se consumo.
- **taglio** $\rightarrow$ **significato intuitivo** $\rightarrow$ interazione fra 2 programmi:
 - uno che produce A
 - l'altro che consuma A

La regole del cut può essere messa in relazione con il Modus Ponens:

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B}$$

- Consumo qualcosa di tipo A per produrre qualcosa di tipo B
- Ho un qualcosa di tipo A
- Ottengo qualcos'altro di tipo B

Va osservato come il tipo A non compaia più, esattamente come la risorsa A non compare più con l'applicazione del taglio.

Nota: in logica lineare è fondamentale che i sequenti siano **multi-nsiemi** di formule, in modo da tenere traccia di occorrenze multiple della medesima risorsa.

Implicazione lineare

$$A \multimap B \equiv A^\top \wp B$$

Significato:

- consuma A e produco B nell'ordine che voglio $\rightarrow$ concettualmente sensato, è così anche con le funzioni in informatica.
- questo operatore viene chiamato *lollipop*

Coimplicazione lineare

$$A \multimap \circ B \equiv (A \multimap B) \otimes (B \multimap A)$$

Tentativo di dimostrazione per $A \multimap (A \multimap B) \multimap A \otimes B$

Tentativo 1 (Fallisce su A)

$$\frac{\frac{\frac{}{\vdash A^\perp, A} \text{ax}}{\vdash A^\perp, A \otimes B^\perp, A \otimes B} \otimes \quad \frac{\frac{?}{\vdash A} \quad \frac{}{\vdash B^\perp, B} \text{ax}}{\vdash B^\perp, A \otimes B} \otimes}{\vdash A^\perp \wp(A \otimes B^\perp) \wp(A \otimes B)} \otimes \wp$$

Tentativo 2 (Fallisce su A)

$$\frac{\frac{\frac{}{\vdash A^\perp, A} \text{ax}}{\vdash A^\perp, A \otimes B^\perp, A \otimes B} \otimes \quad \frac{\frac{?}{\vdash A} \quad \frac{}{\vdash B^\perp, B} \text{ax}}{\vdash A \otimes B^\perp, B} \otimes}{\vdash A^\perp \wp(A \otimes B^\perp) \wp(A \otimes B)} \otimes \wp$$

Il fatto di non riuscire a derivare questa formula in logica lineare è un bene $\rightarrow$ è la formula corrispondente al teorema $A \rightarrow (A \rightarrow B) \rightarrow A \wedge B$ in logica classica.

Tentativo di dimostrazione per $A \multimap (A \multimap B) \multimap A \& B$

Analogia: distributore di caffè che nel caso l'utente non volesse più il caffè dopo aver inserito la moneta, può fornire un rimborso.

$$\frac{\frac{\frac{}{\vdash A^\perp, A} \text{ax} \quad \frac{?}{\vdash B^\perp, A}}{\vdash A^\perp, A \otimes B^\perp, A} \otimes \quad \frac{\frac{}{\vdash A^\perp, A} \text{ax} \quad \frac{}{\vdash B^\perp, B} \text{ax}}{\vdash A^\perp, A \otimes B^\perp, B} \otimes}{\vdash A^\perp, A \otimes B^\perp, A \& B} \& \wp$$

La dimostrazione ad un certo punto fallisce in quanto si osserva che viene prodotto A , ma è solo possibile la consumazione di B .

Quando si ha $\vdash A^\perp, A \otimes B^\perp, A$, $A^\top$ indica la moneta incamerata dal distributore, A indica la moneta che l'utente vuole indietro come rimborso e $A \otimes B^\perp$ indica il processo che trasforma la moneta nel caffè.

Non è possibile concludere la dimostrazione in quanto, se avviene il rimborso, non viene utilizzato il processo di realizzazione del caffè a partire dalla moneta (il processo è un rirsorsa). L'idea è quindi quella di fornire un'alternativa a questo processo tramite **scelta esterna** ($\&$).

Dimostrazione di $A \multimap (1 \& (A \multimap B)) \multimap A \& B$

$$\begin{array}{c}
\frac{\frac{\frac{}{\vdash A^\perp, A} \text{ax}}{\vdash A^\perp, \perp, A} \perp}{\vdash A^\perp, \perp \oplus (A \otimes B^\perp), A} \oplus \quad \frac{\frac{\frac{}{\vdash A^\perp, A} \text{ax} \quad \frac{}{\vdash B^\perp, B} \text{ax}}{\vdash A^\perp, A \otimes B^\perp, B} \otimes}{\vdash A^\perp, \perp \oplus (A \otimes B^\perp), B} \oplus \\
\hline
\vdash A^\perp, \perp \oplus (A \otimes B^\perp), A \& B \\
\hline
\vdash A^\perp \wp (\perp \oplus (A \otimes B^\perp)) \wp (A \& B) \quad \wp \\
\hline
\vdash A^\perp \wp (1 \& (A \multimap B))^\perp \wp (A \& B) \quad \equiv
\end{array}$$

Proprietà della logica lineare

Teorema (eliminazione dei tagli) Se $\vdash \Gamma$ è dimostrabile, esista una prova per $\vdash \Gamma$ che non usa la regola **cut**.

Corollario $\vdash 0$ non è dimostrabile. La logica è consistente.

Dimostrazione Se $\vdash 0$ fosse dimostrabile, lo sarebbe anche senza cut per il teorema precedente. Significa quindi che è stata utilizzata la regola di introduzione di 0 , la quale non esiste $\rightarrow$ assurdo.

π -calcolo

Si tratta di un modello di calcolo per la concorrenza. La versione mostrata dal prof è leggermente diverso da quello originale.

Questo formalismo si basa sul concetto di **canale** $\rightarrow$ mezzo di scambio per i messaggi.

Sintassi

$P, Q ::= x[]$	invio segnale
$ x().P$	ricezione segnale
$ x\langle y \rangle.P$	invio il canale y su x
$ x(y).P$	ricevo canale y su x
$ x \triangleleft \mathbf{b}.P$	invio booleano $\mathbf{b}$ su canale
$ x \triangleright \{P, Q\}$	scelta ($\mathbf{b} \in \mathbf{inl}, \mathbf{inr}$)
$ P \mid Q$	composizione parallela
$ (x)P$	restrizione

- Questa sintassi descrive i **processi** che sono le entità che calcolano e comunicano tramite **canali**.
- Il **segnale** è semplicemente un messaggio vuoto, quando il processo in attesa del segnale lo riceve, prosegue con la propria esecuzione.
- Come possibile osservare dalla sintassi mostrata, la composizione in π -calcolo è parallela, questo rappresenta una grossa differenza con il λ -calcolo.

- la **restrizione** agisce come un canale privato riservato al processo P .

Esempio restrizione

Nel processo $(x)P \mid x[]$ le due x sono due variabili diverse. La prima è riservata a P e significa che ogni occorrenza di x in P indicherà il canale creato con la restrizione.

Nomi liberi

$$\begin{aligned}
\text{fn}(x[]) &= \{x\} \\
\text{fn}(x().P) &= \{x\} \cup \text{fn}(P) \\
\text{fn}(x\langle y \rangle.P) &= \{x, y\} \cup \text{fn}(P) \\
\text{fn}(x(y).P) &= (\{x\} \cup \text{fn}(P)) \setminus \{y\} \\
\text{fn}(x \triangleleft \mathbf{b}.P) &= \{x\} \cup \text{fn}(P) \\
\text{fn}(x \triangleright \{P, Q\}) &= \{x\} \cup \text{fn}(P) \cup \text{fn}(Q) \\
\text{fn}(P \mid Q) &= \text{fn}(P) \cup \text{fn}(Q) \\
\text{fn}((x)P) &= \text{fn}(P) \setminus \{x\}
\end{aligned}$$

Alcuni esempi di processi

Costanti booleane

$$\text{True}(x) = x \triangleleft \mathbf{inl}.x[]$$

$$\text{False}(x) = x \triangleleft \mathbf{inr}.x[]$$

Le costanti booleane vengono realizzate da processi che inviano un booleano e poi inviano un segnale di terminazione.

Costrutto not

$$\text{Not}(x, y) = x \triangleright \{x().\text{False}\langle y \rangle, x().\text{True}\langle y \rangle\}$$

Utilizzando le costanti booleane implementate, è facilmente definibile l'operatore **not**, che nega il booleano ricevuto su x , tramite la **scelta** e lo invia su y

Costrutto copy Analogamente è possibile definire un operatore **copy** che produce su y quel che riceve su x

$$\text{Copy}(x, y) = x \triangleright \{x().\text{True}\langle y \rangle, x().\text{False}\langle y \rangle\}$$

Oppure si può dare una definizione alternativa facendo uso di **restrizione** e **composizione parallela**.

$$\text{Copy}(x, y) = (z)(\text{Not}(x, z) \mid \text{Not}(z, y))$$

Qui la restrizione è fondamentale $\rightarrow$ se non la usassi avrei una **race condition** sia in lettura che in scrittura $\rightarrow$ dal momento che il canale non sarebbe privato, chiunque potrebbe inviare e/o leggere valori booleani.

Costrutto and In questo caso l'operatore riceve in input tre canali $\rightarrow$ il risultato della congiunzione dei primi due viene spedito in output sul terzo.

$$\text{And}(x, y, z) = x \triangleright \{x().\text{Copy}\langle y, z \rangle, x().\text{False}\langle z \rangle\}$$

Semantica operativa del π -calcolo

Riduzioni di base Operazioni complementari e sintatticamente vicine possono interagire:

1. $x[] \mid x().P \rightarrow P$
2. $x \triangleleft \text{inl}.P \mid x \triangleright \{Q, R\} \rightarrow P \mid Q$
3. $x\langle y \rangle.P \mid x(z).Q \rightarrow P \mid Q\{y/z\}$

Chiusura per contesti delle riduzioni

$$\begin{aligned} P \mid Q &\rightarrow P' \mid Q && \text{se } P \rightarrow P' \\ P \mid Q &\rightarrow P \mid Q' && \text{se } Q \rightarrow Q' \\ (x)P &\rightarrow (x)P' && \text{se } P \rightarrow P' \end{aligned}$$

Esempio $x().P \mid x[] \not\rightarrow$ con le regole attuali, tuttavia non va bene poiché la composizione, essendo parallela, dovrebbe essere **commutativa** (differenza con λ -calcolo).

Esempio $x[] \mid (y[] \mid x().P)$: in questo altro esempio si osserva che $x[]$ e $x().P$ potrebbero sincronizzarsi, ma non possono in quanto sintatticamente non vicini.

La soluzione a queste problematiche è data dalla **congruenza strutturale**.

Congruenza strutturale Consiste in una **riscrittura sintattica** dei processi per consentire tutte le sincronizzazioni possibili.

Si usa il simbolo $\equiv$ per indicare la più piccola **congruenza** tale che:

$$\begin{aligned} P \mid Q &\equiv Q \mid P \\ P \mid (Q \mid R) &\equiv (P \mid Q) \mid R \\ (x)P \mid Q &\equiv (x)(P \mid Q) && x \notin \text{fn}(Q) \\ (x)P &\equiv P && x \notin \text{fn}(P) \end{aligned}$$

La congruenza è una **relazione di equivalenza**. La terza regola significa che posso portare Q sotto la restrizione di x se x non compare libera in Q . La quarta invece indica che se il canale introdotto dalla restrizione non viene usato, allora posso liberarmi della restrizione.

Esempi di riduzione

1. Esempio di riduzione in cui non serve usare la **congruenza strutturale** in quanto a ogni passo gli elementi che possono interagire sono sempre **sintatticamente vicini**.

$$\begin{aligned} \text{True}\langle x \rangle \mid \text{Not}\langle x, y \rangle &= x \triangleleft \mathbf{inl}.x[] \mid x \triangleright \{x().\text{False}\langle y \rangle, x().\text{True}\langle y \rangle\} \\ &\rightarrow x[] \mid x().\text{False}\langle y \rangle \\ &\rightarrow \text{False}\langle y \rangle \end{aligned}$$

2. Esempio semplice

$$\text{False}\langle x \rangle \mid \text{Not}\langle x, y \rangle \rightarrow^2 \text{True}\langle y \rangle$$

3. Esempio che richiede l'utilizzo della **congruenza strutturale**

$$\begin{aligned} \text{True}\langle x \rangle \mid \text{Copy}\langle x, y \rangle &= \text{True}\langle x \rangle \mid (z)(\text{Not}\langle x, z \rangle \mid \text{Not}\langle z, y \rangle) \\ &\equiv (z)((\text{True}\langle x \rangle \mid \text{Not}\langle x, z \rangle) \mid \text{Not}\langle z, y \rangle) \\ &\rightarrow^2 (z)(\text{False}\langle z \rangle \mid \text{Not}\langle z, y \rangle) \\ &\rightarrow^2 (z)\text{True}\langle y \rangle \\ &\equiv \text{True}\langle y \rangle \end{aligned}$$

Nel passaggio 1 si nota subito che non è possibile procedere con una riduzione. Viene quindi applicata la terza regola della congruenza in quanto z non compare libera in $\text{True}\langle x \rangle \equiv x \triangleleft \mathbf{inl}.x[]$.

Anche nel passaggio 4 devo utilizzare la congruenza $\rightarrow$ in particolare viene applicata la quarta regola che consente di rimuovere la restrizione dal momento che z è inutilizzato.

Sessioni

Si vuole modellare un sistema in cui Bob vuole inviare due informazioni, u e v ad Alice o a Carol, in modo che entrambe le info siano ricevute da solo na delle due attrici.

Prima versione

$$\begin{aligned} \text{Bob}(x, u, v) &= x\langle u \rangle.x\langle v \rangle.P & \text{Alice}(x) &= x(y).x(z).Q \\ & & \text{Carol}(x) &= x(y).x(z).R \end{aligned}$$

Lo schema della comunicazione è dato dal seguente processo:

$$(\text{Bob}\langle x, u, v \rangle \mid \text{Alice}\langle x \rangle) \mid \text{Carol}\langle x \rangle$$

Tuttavia riducendo questo processo si raggiungono 4 possibili configurazioni finali:

- Alice riceve entrambi i messaggi
- Carolo riceve entrambi i messaggi
- 2 configurazioni in cui entrambe la attrici ricevono un messaggio a testa
→ **configurazioni indesiderate**

Il problema risiede nell'utilizzo di un **unico canale pubblico** → soluzione: utilizzo di canali privati per realizzare delle **sessioni**.

Versione con sessioni

$$\begin{aligned} \text{Bob}(x, u, v) &= (s)(x\langle s \rangle.s\langle u \rangle.s\langle v \rangle.P) & \text{Alice}(x) &= x(t).t(y).t(z).Q \\ & & \text{Carol}(x) &= x(t).t(y).t(z).R \end{aligned}$$

L'introduzione della restrizione nel processo Bob è ciò che permette di realizzare un canale privato che è la sessione. Questo nuovo canale viene poi inviato su x , che è l'unico ricevente della sessione → chi riceve s ricercherà quindi entrambe le informazioni.

Nota: potrebbe sembrare che chi non riceve rimanga in starvation, in realtà non è così in quanto rimane in attesa di ricevere la sessione su un canale **pubblico** → chiunque potrebbe inviare un messaggio.

Comportamenti non desiderati

Si tratta di processi sintatticamente corretti, ma che danno origine a situazioni indesiderate:

- $(x)(x().P)$: **starvation** → perché il canale è privato
- $(x)(x[])$: **messaggio orfano**
- $x \triangleleft \text{inl}.P \mid x(y).Q$: **errore di comunicazione**
- $x \triangleleft \text{inl}.P \mid x \triangleleft \text{inr}.Q \mid x \triangleright \{R_1, R_2\}$: **race condition in scrittura**
- $x \triangleleft b.P \mid x \triangleright \{Q_1, Q_2\} \mid x \triangleright \{R_1, R_2\}$: **race condition in lettura**
- $(x)(y)(x().y[] \mid y().x[])$: **deadlock**

Nota per esercizi: quando si fanno gli esercizi di codifica con π -calcolo, bisogna assicurarsi di usare tutte le risorse in input, in ogni ramo della computazione.

Esempio

$$\text{Or}(x, y, z) = x \triangleright \{x().y \triangleright \{y().\text{True}\langle z \rangle, y().\text{True}\langle z \rangle\}, x().\text{Copy}\langle y, z \rangle\}$$

Questa implementazione garantisce un uso **lineare** delle risorse $\rightarrow$ si parla di logiche non più lineari, ma **affini**.

CP

CP (Classical Process) è il calcolo per la concorrenza che è stato messo in relazione con la **logica lineare**.

π -calcolo	CP	Significato
$P, Q ::= -$	$x \leftrightarrow y$	link
$-$	$x \triangleright \{\}$	errore
$x[]$	$x[]$	invio segnale
$x().P$	$x().P$	ricevo segnale
$x\langle y \rangle.P$	$-$	invio canale esistente
$-$	$x[y](P \mid Q)$	invio canale nuovo
$x(y).P$	$x(y).P$	ricevo canale
$x \triangleright \{P, Q\}$	$x \triangleright \{P, Q\}$	invio $b \in \{\mathbf{inl}, \mathbf{inr}\}$
$P \mid Q$	$-$	composizione parallela
$(x)P$	$-$	restrizione
$-$	$(x)(P \mid Q)$	cut

I costrutti del π -calcolo $P \mid Q$ e $(x)P$ vengono uniti in un unico costrutto in CP: $(x)(P \mid Q)$ che è più vincolante $\rightarrow$ se componi in parallelo deve esserci una comunicazione tra i processi in parallelo. Il costrutto $x\langle y \rangle.P$ diventa $x[y](P \mid Q)$ in CP $\rightarrow$ uno dei due processi rappresenta come viene gestito il messaggio, l'altro rappresenta la continuazione del processo. Il costrutto $x \triangleright \{\}$ rappresenta un processo in attesa di un messaggio che non arriverà mai, arriva dalla logica $\rightarrow$ regola per $\top$. Il costrutto $x \leftrightarrow y$ invece modella un processo che agisce come un forwarder $\rightarrow$ lettura da un canale e inoltra sull'altro (**bidirezionale**).

Nonostante CP non fornisca l'operatore $x\langle y \rangle.P$ è comunque possibile otterlo:

$$x\langle y \rangle.P = x[z](z \leftrightarrow y \mid P)$$

Nomi liberi

$\text{fn}(P) \rightarrow$ nomi che occorrono **liberi** in P :

$$\begin{aligned}
\text{fn}(x \leftrightarrow y) &= \{x, y\} \\
\text{fn}(x \triangleright \{\}) &= \{x\} \\
\text{fn}(x[y](P \mid Q)) &= (\text{fn}(P) \setminus \{y\}) \cup \text{fn}(Q) \\
\text{fn}((x)(P \mid Q)) &= (\text{fn}(P) \cup \text{fn}(Q)) \setminus \{x\}
\end{aligned}$$

L'insieme dei nomi legati in P viene indicato con $\text{bn}(P)$

Semantica operativa

Anche in CP, operazioni **complementari** devono essere sintatticamente vicine per interagire.

$$\begin{aligned}
(x)(x \leftrightarrow y \mid P) \mathbb{B}P\{y/x\} \\
(x)(x[] \mid x().P) &\rightarrow P \\
(x)(x \triangleleft \mathbf{inl}.P \mid x \triangleright \{Q, R\}) &\rightarrow (x)(P \mid Q) \\
(x)(x[y](P \mid Q) \mid x(z).R) &\rightarrow (y)(P \mid (x)(Q \mid R\{y/z\}))
\end{aligned}$$

Chiusura per contesti

$$(x)(P \mid Q) \rightarrow (x)(P' \mid Q) \quad \text{se } P \rightarrow P'$$

Regola simmetrica omessa perché il parallelo è commutativo.

Congruenza strutturale

Si usa il simbolo $\equiv$ per indicare la più piccola congruenza tale che:

$$\begin{aligned}
x \leftrightarrow y &\equiv y \leftrightarrow x \\
(x)(P \mid Q) &\equiv (x)(Q \mid P) \\
(x)(P \mid (y)(Q \mid R)) &\equiv (y)((x)(P \mid Q) \mid R) \quad x \notin \text{fn}(R), y \notin \text{fn}(P)
\end{aligned}$$

Chiusura per contesti

$$P \rightarrow Q \quad \text{se } P \equiv R \text{ e } R \rightarrow Q$$

Corrispondenza Curry-Howard per CP

CP	Logica lineare
tipi	proposizioni
processi	prove
congruenza	equivalenza tra prove

CP	Logica lineare
riduzione	semplificazione di prove

I tipo vengono **associati ai canali**, non ai processi $\rightarrow$ grossa differenza con λ -calcolo. L'intero sequente può essere visto come il tipo del processo.

Dai sequenti ai giudizi di tipo

- I sequenti vengono arricchiti da un **contesto di tipo** $\rightarrow$ mappa finita da canali a tipi, indicati con Γ, Δ .
- $x : A$ è il contesto singoletto che associa A ad x .
- Γ, Δ indica l'**unione disgiunta** di Γ e Δ
- Γ, Δ definito sse $\text{dom}(\Gamma) \cap \text{dom}(\Delta) = \emptyset$

Regole di tipo

Le seguenti regole di tipaggio sono in corrispondenza 1 a 1 con le regole

Costanti

$$\frac{}{x[] \vdash x : \mathbf{1}} \quad \frac{}{x \triangleright \{\} \vdash \Gamma, x : \top} \quad \frac{P \vdash \Gamma}{x().P \vdash \Gamma, x : \perp} \quad \text{nessuna regola per } 0$$

Connettivi moltiplicativi

$$\frac{P \vdash \Gamma, y : A \quad Q \vdash \Delta, x : B}{x[y](P \mid Q) \vdash \Gamma, \Delta, x : A \otimes B} \quad \frac{P \vdash \Gamma, y : A, x : B}{x(y).P \vdash \Gamma, x : A \wp B}$$

Connettivi additivi (contesto condiviso)

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Gamma, x : B}{x \triangleright \{P, Q\} \vdash \Gamma, x : A \& B} \quad \frac{P \vdash \Gamma, x : A}{x \triangleleft \mathbf{inl}.P \vdash \Gamma, x : A \oplus B} \quad \frac{P \vdash \Gamma, x : B}{x \triangleleft \mathbf{inr}.P \vdash \Gamma, x : A \oplus B}$$

Altre regole

$$\frac{}{x \leftrightarrow y \vdash x : A, y : A^\perp} \text{ assioma} \quad \frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, x : A^\perp}{(x)(P \mid Q) \vdash \Gamma, \Delta} \text{ taglio (o cut)}$$

La regola di tipaggio del cut mi dice che P e Q per comunicare devono usare un **unico canale privato** $\rightarrow$ impossibile avere deadlock.

Proprietà indesiderate Le proprietà indesiderate mostrate precedentemente non sono più possibili $\rightarrow$ non tipano, sintatticamente errate. Le regole di tipaggio **non perettono** errori di comunicazione.

Sistema di tipi lineare Un sistema di tipi lineare (o *sub-strutturale*) è un sistema in cui il tipo assegnato alla variabile descrive un **protocollo**. Usare due o più volte la medesima variabile, viola il protocollo.

Preservazione del tipaggio rispetto a $\equiv$

Lemma

Se $P \equiv Q$ allora $P \vdash \Gamma$ implica $Q \vdash \Gamma$.

Significa che due processi equivalenti, avranno canali con gli stessi tipi.

Dimostrazione La dimostrazione avviene per induzione sulla prova di $P \equiv Q$. La dimostrazione si concentra sui 3 casi base delle regole di congruenza, i casi induttivi sono più semplici.

- **caso** $P = x \leftrightarrow y \equiv y \leftrightarrow x = Q$. Dall'assioma e usando il fatto che $^\perp$ è un'involuzione, si deduce $\Gamma = x : A, y : A^\perp = y : A^\perp, x : A^{\perp\perp}$. Qui si applica un'istanza dell'assioma e si ottiene $y \leftrightarrow x$.
- **caso** $P = (x)(P_1 \mid P_2) \equiv (x)(P_2 \mid P_1) = Q$. Dalla regola di cut si deduce $\Gamma = \Gamma_1, \Gamma_2$ e $P_i \vdash \Gamma_i, x : A_i$ con $A_1 = A_2^\perp$, ovvero $A_2^\perp = A_1^{\perp\perp}$. Si conclude con un'applicazione della regola cut.
- **caso** $P = (x)(S \mid (y)(T \mid R)) \equiv (y)((x)(S \mid T) \mid R) = Q$. Dalla regola cut e dall'ipotesi $y \notin \text{fn}(S)$ si deduce che $S \vdash \Gamma_1, x : A$ e che $(y)(T \mid R) \vdash \Gamma_2, x : A^\perp$. Dalla regola cut e dall'ipotesi $x \notin \text{fn}(R)$ si ha che $T \vdash \Gamma_3, y : B, x : A^\perp$ e che $R \vdash \Gamma_4, y : B^\perp$.

Posso quindi applicare un'istanza di cut a S e T , ottenendo $(x)(S \mid T) \vdash \Gamma_1, \Gamma_3, y : B$. Applicando una seconda istanza di cut si ottiene $(y)((x)(S \mid T) \mid R) \vdash \Gamma_1, \Gamma_3, \Gamma_4$. Dal momento che $\Gamma = \Gamma_1, \Gamma_3, \Gamma_4$, la dimostrazione è conclusa: $Q \vdash \Gamma$.

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

La dimostrazione è abbastanza lunga in quanto deve coprire tutti i possibili casi induttivi per ciascuna delle tre leggi dell'equivalenza, tuttavia si tratta di casi in cui è solamente necessario applicare meccanicamente l'ipotesi induttiva.

Preservazione del tipaggio rispetto a $\rightarrow$

Lemma

Se $P \vdash \Gamma, x : A$ e $y \notin \Gamma$ allora $P\{y/x\} \vdash \Gamma, y : A$

Teorema

Se $P \rightarrow Q$ allora $P \vdash \Gamma$ implica $Q \vdash \Gamma$.

I canali di un processo hanno tipo uguale ai canali di un processo in cui esso si riduce.

Dimostrazione La dimostrazione procede per induzione sulla derivazione di $P \rightarrow Q$, vengono mostrati solo i quattro casi base:

- **caso** $P = (x)(x \leftrightarrow y \mid R) \rightarrow R\{y/x\} = Q$. Da $P \vdash \Gamma$ e dal fatto che è stata applicata la regola cut, si deduce:

- $\Gamma = \Gamma_1, \Gamma_2$
- $x \leftrightarrow y \vdash \Gamma_1, x : A$
- $R \vdash \Gamma_2, x : A^\perp$

Dall'assioma, se $x \leftrightarrow y \vdash x : A, \Gamma_1$ allora Γ_1 deve avere forma $y : A^\perp$. Si vuole dimostrare $R\{y/x\} \vdash \Gamma$. Sapendo che $R \vdash \Gamma_2, x : A^\perp$, si applica il lemma e si ottiene $R\{y/x\} \vdash \Gamma_2, y : A^\perp$, ma $y : A^\perp = \Gamma_1$ da cui $R\{y/x\} \vdash \Gamma$.

- **caso** $P = (x)(x[] \mid x().Q) \rightarrow Q$. Da $P \vdash \Gamma$ e dal fatto che è stata applicata la regola cut, si deduce:

- $\Gamma = \Gamma_1, \Gamma_2$
- $x[] \vdash x : \mathbf{1}$
- $x().P \vdash \Gamma_2, x : \perp$

Dalla regola $\mathbf{1}$, se $x[] \vdash x : \mathbf{1}$ allora $\Gamma_1 = e$ e $A = \mathbf{1}$. Dalla regola $\perp$ si deduce $Q \vdash \Gamma_2$. Dal momento che Γ_1 è vuoto, $Q \vdash \Gamma$.

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

- **caso** $P = (x)(x \triangleleft \mathbf{inl}.T \mid x \triangleright \{Q, R\}) \rightarrow (x)(T \mid Q) = S$. Da $P \vdash \Gamma$ e dall'applicazione di cut:

- $\Gamma = \Gamma_1, \Gamma_2$
- $x \triangleleft \mathbf{inl}.T \vdash \Gamma_1, x : A^\perp \oplus B^\perp$
- $x \triangleright \{Q, R\} \vdash \Gamma_2, x : A \& B$

Dalla regola $\&$ si deduce che $Q \vdash \Gamma_2, x : A$ e $R \vdash \Gamma_2, x : B$. Dalla regola $\oplus$ si deduce $T \vdash \Gamma_1, x : A^\perp$. Applicando la regola cut a $(x)(T \mid Q)$, si ottiene $S \vdash \Gamma_1, \Gamma_2$, ovvero $S \vdash \Gamma$.

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

- **caso** $P = (x)(x[y](T \mid Q) \mid x(z).R) \rightarrow (y)(T \mid (x)(Q \mid R\{y/z\})) = S$. Da $P \vdash \Gamma$ e dall'applicazione di cut:

- $\Gamma = \Gamma_1, \Gamma_2$
- $x[y](T \mid Q) \vdash \Gamma_1, x : A^\perp \otimes B^\perp$
- $x(z).R \vdash \Gamma_2, x : A \wp B$

Dall'applicazione della regola $\otimes$ si deduce che $\Gamma_1 = \Gamma_3, \Gamma_4$, $T \vdash \Gamma_3, y : A^\perp$ e che $Q \vdash \Gamma_4, x : B^\perp$. Per la regola $\wp$ invece si ha $R \vdash \Gamma_2, z : A, x : B$. Applicando il lemma precedentemente definito a R , si ottiene che $R\{y/z\} \vdash \Gamma_2, y : A, x : B$. Applicando un'istanza della regola cut su Q e $R\{y/z\}$ si deduce $(x)(Q \mid R\{y/z\}) \vdash \Gamma_2, \Gamma_4, y : A$. Applicando una seconda istanza della regola cut si ottiene $(y)(T \mid (x)(Q \mid R\{y/z\})) \vdash \Gamma_2, \Gamma_3, \Gamma_4$. Dal momento che $\Gamma_1 = \Gamma_3, \Gamma_4$ e $\Gamma = \Gamma_1, \Gamma_2$ si ha $(y)(T \mid (x)(Q \mid R\{y/z\})) \vdash \Gamma$.

Attenzione !!!: questo caso è stato omesso dal prof, dimostrazione mia.

Progresso e terminazione

Nozioni ausiliarie

Thread Un x -thread è un processo che inizia con un'azione **sequenziale** su $x \rightarrow x[], x().P, x(y).P, x \leftrightarrow y, \dots$

L'ultimo è anche un y -thread. In sostanza ogni processo che non sia un cut è un thread.

Nota: un thread **può** contenere un cut, quello che è rilevante è la prima azione.

Contesti di riduzione

$$\mathcal{C}, \mathcal{D} ::= [] \mid (x)(\mathcal{C} \mid P) \mid (x)(P \mid \mathcal{C})$$

Si tratta di processi che hanno un (e uno solamente) *buco*. Nel caso il buco venga riempito da un processo la cui variabile libera viene catturata, non avviene alcuna ridenominazione $\rightarrow$ la cattura è, in questo caso, un effetto desiderato.

La nozione di contesto è importante per decidere quali processi sono da considerarsi effettivamente dei deadlock. Si consideri l'esempio:

$$(x)(z().x[] \mid x().y[]) \not\rightarrow$$

In realtà non si vuole che questo processo sia considerato un deadlock, infatti è bloccato su $z()$ che è un canale pubblico. Utilizzando un contesto $\mathcal{C} = (z)([] \mid z[])$ si riesce a mostrare che il processo iniziale può essere sbloccato.

Lemma di prossimità

Se valgono:

1. $x \in \text{fn}(P) \setminus (\text{fn}(\mathcal{C}) \cup \text{bn}(\mathcal{C}))$
2. $\text{bn}(\mathcal{C}) \cap \text{fn}(Q) = \emptyset$

allora esiste $\mathcal{D}$ tale che

$$(x)(\mathcal{C}[P] \mid Q) \equiv \mathcal{D}[(x)(P \mid Q)]$$

Dimostrazione

La dimostrazione è per induzione strutturale su $\mathcal{C}$:

- **caso** $\mathcal{C} = []$. Con $\mathcal{C} = []$ si ottiene $(x)(P \mid Q)$. La dimostrazione del caso si conclude scegliendo $\mathcal{D} = []$.
- **caso** $\mathcal{C} = (y)(\mathcal{C}' \mid R)$.

Per la prima ipotesi:

- $x \neq y$
- $x \notin \text{fn}(R)$
- $x \in \text{fn}(P) \setminus (\text{fn}(\mathcal{C}') \cup \text{bn}(\mathcal{C}'))$

Per la seconda ipotesi:

- $y \notin \text{fn}(Q)$
- $\text{bn}(\mathcal{C}') \cap \text{fn}(Q) =$

Per ipotesi induttiva esiste $\mathcal{D}'$ tc $(x)(\mathcal{C}'[P] \mid Q) \equiv \mathcal{D}'[(x)(P \mid Q)]$. Si ottiene:

$$\begin{aligned} (x)(\mathcal{C}[P] \mid Q) &= (x)((y)(\mathcal{C}'[P] \mid R) \mid Q) \\ &\equiv (x)(Q \mid (y)(\mathcal{C}'[P] \mid R)) \\ &\equiv (y)((x)(Q \mid \mathcal{C}'[P]) \mid R) \quad x \notin \text{fn}(R), y \notin \text{fn}(Q) \\ &\equiv (y)((x)(\mathcal{C}'[P] \mid Q) \mid R) \\ &\equiv (y)(\mathcal{D}'[(x)(P \mid Q)] \mid R) \end{aligned}$$

La dimostrazione si conclude scegliendo $\mathcal{D} = (y)(\mathcal{D}' \mid R)$.

- **caso** $\mathcal{C} = (y)(R \mid \mathcal{C}')$. Analogo al soprastante.

Teorema: Progresso

Se $P \vdash \Gamma$ allora:

1. esiste Q tc $P \rightarrow Q$ oppure
2. esistono $\mathcal{C}$ e un x -thread Q tali che $P = \mathcal{C}[Q]$ e $x \notin \text{bn}(\mathcal{C})$.
Sostanzialmente si tratta di un processo in attesa di interagire con l'ambiente esterno.

Dimostrazione

La dimostrazione avviene per induzione strutturale su P .

- **caso P è un x -thread.** Prendendo $\mathcal{C} = []$ e $Q = P$, osservando che $x \in \text{dom}(\Gamma)$.
- **caso $P = (x)(P_1 \mid P_2)$.** Per il tipaggio è stata sicuramente applicata la regola di cut, segue che $\Gamma = \Gamma_1, \Gamma_2$ e $P_i \vdash \Gamma_i, x : A_i$, dove $A_1 = A_2^\perp$.

Applicando entrambe le ipotesi induttive ($i = 1, 2$) si deduce che:

1. esiste Q_i tc $P_i \rightarrow Q_i$ oppure
2. esistono $\mathcal{C}_i$ e un x_i -thread Q_i tali che $P_i = \mathcal{C}_i[Q_i]$ e $x_i \notin \text{bn}(\mathcal{C}_i)$.

Le due possibilità unite alle due ipotesi induttive creano uno scenario in cui ci sono quattro possibili sottocasi:

- **caso 1 dell'ipotesi induttiva 1.** Prendendo $Q = (x)(Q_1 \mid P_2)$, osservando che $P \rightarrow Q$, il caso è dimostrato. Se $P_1 \rightarrow Q_1$, allora $P \rightarrow Q$ per come è stato definito Q , ma allora P si riduce.
- **caso 1 dell'ipotesi induttiva 2.** Prendendo $Q = (x)(P_1 \mid Q_2)$, osservando che $P \rightarrow Q$, il caso è dimostrato.
- **caso 2 per entrambe le ipotesi induttive.**

Si ha: $P_1 = \mathcal{C}_1[Q_1]$ e $P_2 = \mathcal{C}_2[Q_2]$. Q_1 e Q_2 sono rispettivamente un x_1 -thread e un x_2 -thread, tali che $x_1 \notin \text{bn}(\mathcal{C}_1)$ e $x_2 \notin \text{bn}(\mathcal{C}_2)$

In questo caso occorre analizzare degli ulteriori sotto-casi:

- * **caso $x_1 \neq x$.** Si conclude prendendo $\mathcal{C} = (x)(\mathcal{C}_1 \mid \mathcal{P}_2)$, osservando che $x_1 \notin \text{bn}(\mathcal{C})$. $\mathcal{P}$ soddisfa la condizione 2.
- * **caso $x_2 \neq x$.** Analogo al soprastante.
- * **caso $x_1 = x_2 = x$.** Q_1 e Q_2 sono degli x -thread.

$$\begin{aligned}
P &= (x)(P_1 \mid P_2) \\
&= (x)(\mathcal{C}_1[Q_1] \mid \mathcal{C}_2[Q_2]) \\
&\equiv \mathcal{D}_1[(x)(Q_1 \mid \mathcal{C}_2[Q_2])] && \text{Lemma di prossimità} \\
&\equiv \mathcal{D}_1[(x)(\mathcal{C}_2[Q_2] \mid Q_1)] \\
&\equiv \mathcal{D}_2[\mathcal{D}_1[(x)(Q_2 \mid Q_1)]] && \text{Lemma di prossimità} \\
&\equiv \mathcal{D}_2[\mathcal{D}_1[(x)(Q_1 \mid Q_2)]]
\end{aligned}$$

Sia Q_1 che Q_2 sono x -thread e sotto processi immediati di un cut ben-tipato su x . Segue che se i due thread sono fra loro compatibili, allora il processo si riduce e la dimostrazione è conclusa. Occorre quindi dimostrare che Q_1 e Q_2 sono compatibili secondo le regole della semantica operazione strutturata.

Dal momento che il cut è ben tipato $\rightarrow Q_1$ e Q_2 usano il canale x in modo complementare, dunque P

Teorema: Terminazione

Per ogni P , non esiste una sequenza infinita di riduzioni $P \rightarrow P_1 \rightarrow P_2 \rightarrow \dots$

Dimostrazione

Sia $|P|$ il naturale definito per induzione sulla struttura di P come segue:

$$\begin{aligned} |x \leftrightarrow y| &= 1 \\ |x[]| &= 1 \\ |x().P| &= 1 + |P| \\ \vdots &= \vdots \\ |(x)(P \mid Q)| &= 1 + |P| + |Q| \end{aligned}$$

Basta constatare che $P \equiv Q$ implica $|P| = |Q|$ e $P \rightarrow Q$ implica $|Q| < |P|$ (ogni riduzione diminuisce la dimensione del processo).

Cut elimination per MALL**Ammissibilità del cut**

Una regola di inferenza si dice **ammissibile** se la sua presenza non altera l'insieme dei sequenti che possono essere derivati.

Teorema

La regola cut in MALL è ammissibile.

Riduzioni profonde

Si vuole ridurre il processo il più possibile per semplificare la dimostrazione, sfruttando la **subject reduction**. Esempio:

$$\begin{aligned} (y)(y[]|(z)(y().z[]|z().x[])) &\vdash x : \mathbf{1} \\ \equiv \\ (z)((y)(y[]|y().z[])|z().x[]) &\vdash x : \mathbf{1} \\ \downarrow \\ (z)(z[]|z().x[]) &\vdash x : \mathbf{1} \\ \downarrow \\ x[] &\vdash x : \mathbf{1} \end{aligned}$$

Tuttavia per potere verificare il buon tipaggio dei processi possibili, occorre poter ridurre processi bloccati da un prefisso. Esempio: $x().(z)(z[] \mid z().y[]) \vdash x : \perp, y :$

1. Il processo mostrato nell'esempio infatti, compie un solo passo di riduzione in $x().y[] \vdash x : \perp, y : \mathbf{1}$.

Una prima soluzione potrebbe essere quella di permettere le riduzioni ovunque, invece che solo nei cut:

$$\frac{P \rightarrow Q}{x().P \rightarrow x().Q} \quad \frac{P \rightarrow Q}{x(y).P \rightarrow x(y).Q} \quad \dots$$

Le proprietà di **terminazione** e **subject reduction** continuano a valere.

Il problema della soluzione Il controesempio $(z)(x().z[] \mid z().y[]) \vdash x : \perp, y : \mathbf{1}$ non si riduce in alcun modo $\rightarrow$ il prefisso $x()$ blocca la riduzione su z . Nonostante ciò una dimostrazione senza cut di $x : \perp, y : \mathbf{1}$ esiste:

$$x().y[] \vdash x : \perp, y : \mathbf{1}$$

Azioni interne ed esterne

Osservando il processo:

$$(z)(x().z[] \mid z().y[]) \vdash x : \perp, y : \mathbf{1}$$

1. Apertura sessione $z \rightarrow$ si tratta di un'azione **interna** perché z è legato
2. Attesa di un segnale da $x \rightarrow$ si tratta di un'azione **esterna** perché x è libero
3. Chiusura sessione $z \rightarrow$ si tratta di un'azione **interna** perché z è legato
4. Invio segnale su $y \rightarrow$ si tratta di un'azione **esterna** perché y è legato

Osservazione: per chi interagisce con questo processo è **irrilevante** che la sessione z venga aperta prima o dopo la ricezione del canale su x . Il canale x non è legato ed è ciò che blocca il processo.

Commuting conversion

Si tratta di un'estensione della congruenza strutturale, per permettere ulteriori permutazioni che non alterano il processo. L'idea è quella di commutare cut e azioni esterne per consentire l'abilitazione di eventuali **riduzioni profonde** precedentemente bloccate. Si usa il simbolo $\equiv_{cc}$ per indicare a più piccola **congruenza** tale che:

$$\begin{array}{ll}
x \leftrightarrow y \equiv_{cc} y \leftrightarrow x & \\
(x)(P \mid Q) \equiv_{cc} (x)(Q \mid P) & \\
(x)(P \mid (y)(Q \mid R)) \equiv_{cc} (y)((x)(P \mid Q) \mid R) & x \notin \text{fn}(R), y \notin \text{fn}(P) \\
(x)(y \triangleright \{ \} \mid P) \equiv_{cc} y \triangleright \{ \} & x \neq y, \text{ da } \text{sx a } \text{dx} \\
(x)(y().P \mid Q) \equiv_{cc} y().(x)(P \mid Q) & x \neq y \\
(x)(y[z](P \mid Q) \mid R) \equiv_{cc} y[z]((x)(P \mid R) \mid Q) & x \in \text{fn}(P) \setminus \{y, z\} \\
(x)(y[z](P \mid Q) \mid R) \equiv_{cc} y[z](P \mid (x)(Q \mid R)) & x \in \text{fn}(Q) \setminus \{y, z\} \\
(x)(y \triangleleft b.P \mid Q) \equiv_{cc} y \triangleleft b.(x)(P \mid Q) & x \neq y \\
(x)(y \triangleright \{P, Q\} \mid R) \equiv_{cc} y \triangleright \{(x)(P \mid R), (x)(Q \mid R)\} & x \neq y
\end{array}$$

Chiusura riduzione rispetto alla congruenza

$$P \rightarrow Q \quad \text{se } P \equiv_{cc} R \text{ e } R \rightarrow Q$$

Proprietà di progresso rivisitata: Teorema

Se $P \vdash \Gamma$ allora:

1. esiste Q tale che $P \rightarrow Q$, oppure
2. esiste un thread Q tale che $P \equiv_{cc} Q$

Idea di dimostrazione

La dimostrazione avviene per induzione sulla struttura di P :

- se P è un thread basta prendere $Q = P$.
- se $P = (x)(P_1 \mid P_2)$ si usa l'ipotesi induttiva su P_1 e P_2 :
 - se P_1 si riduce e/o P_2 si riduce, allora P si riduce
 - se $P_i \equiv_{cc} Q_i$ e i Q_i sono entrambi thread, si distinguono due sotto-casi:
 - * se Q_1 e Q_2 sono entrambi x -thread allora si possono ridurre per la proprietà di **progresso** e dunque P si riduce.
 - * se uno dei due è un y -thread con $x \neq y$, uso $\equiv_{cc}$ per estrarre il prefisso del y -thread fuori dal cut e ottenere il thread Q .

Teorema: cut elimination

Dato $P \vdash \Gamma$ esiste Q senza cut tale che $Q \vdash \Gamma$.

Dimostrazione

La dimostrazione è per induzione su $|P|$.

Dalle proprietà di **terminazione**, **progresso** e **subject reduction** si deduce l'esistenza di un thread R tale che:

$$P \rightarrow P_1 \rightarrow \dots \rightarrow_{\equiv_{cc}} R \quad \text{e} \quad |R| \leq |P| \quad \text{e} \quad R \vdash \Gamma$$

In generale R avrà la forma di un certo prefisso seguito da $[0, \dots, 2]$ continuazioni P_i tale che $|P_i| < |R| \leq |P|$ e $P_i \vdash \Gamma_i$ per un certo Γ_i . Applicando l'ipotesi induttiva si deduce che $\forall P_i \exists Q_i$ senza cut tale che $Q_i \vdash \Gamma_i$. Si ottiene il Q desiderato, andando a rimpiazzare ogni continuazione P_i con il corrispondente Q_i in R .